```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.MobTide.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S07_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
            // S1 tide stage mounts itself in code (G07-T005); S2 scene authoring replaces this.
            var host = new GameObject("TideStageView");
            host.AddComponent&lt;TideStageView&gt;();
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
```

```
namespace Sy.MobTide.Boot
{
    /// &lt;summary&gt;
    /// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
    /// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
```

```
        }
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.MobTide.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// T009: the ads service rides the channel suite (PlatformBootstrap) so
    /// ShowRewarded consumption is category-capped on the shared AdCap in every
    /// channel - the rescue pre-check and the consume can never disagree.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static readonly AdCapPolicy AdCap = new AdCapPolicy();
        /// &lt;summary&gt;Resolved channel suite (T009). Non-null once RegisterAll
        /// ran; the suite's ads service is what ServiceLocator hands out.&lt;/summary&gt;
        public static PlatformSuite Suite { get; private set; }
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
```

```
        ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
        ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
        ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
        // T009: channel suite wiring. PlatformBootstrap gates SY_WECHAT /
        // SY_DOUYIN builds to the real channel suites; editor/batchmode stays
        // mock. Ads consume from the shared AdCap in every channel so the
        // rescue pre-check/consume same-instance law survives the suite switch.
        Suite = PlatformBootstrap.Create(AdCap);
        ServiceLocator.Register&lt;PlatformSuite&gt;(Suite);
        ServiceLocator.Register&lt;IAdsService&gt;(Suite.Ads);
        Logger.Info("BOOT", "services registered (suite=" + PlatformBootstrap.SuiteIdFor(Suite.Channel) + ")");
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
```

```
            return key;
        }
        public FontSet GetFontSet()
            =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
    }
}
// CityRotation.cs
// CityRotation.cs - daily LiveOps city-front rotation for MobTide (G07-T015).
// The siege stands at a different city every day: one of seven embedded
// garrison themes (the config ships in the build - zero server, red line 12)
// rotates on the very day key the ad cap already flips on (one boundary),
// and the featured front city rides the same integer seed (dayseed % 30 -
// the SaveWire city-mask cap), so every device reads the same front on the
// same local day: same day, all ends, one city. The clock is device-local
// degradation: TodayKey() is the single time face and a malformed key
// degrades to seed 0 deterministically - never throws, never calls out.
// Derivation is pure integer math: two derives of one key are byte-identical
// through both the string face and the seed face. The rotation owns only its
// own faces - the front banner line and its accent (TideStageView); the tide
// engine, the wave table and the resolver constants stay byte-identical
// whatever the day brings (RotationCheck proves the event stream never moves).
// Festival window (G07-T032 LiveOps holiday skin): local calendar days
// 10-01..10-08 flip the banner line and its accent to the HOLIDAY TIDE skin -
// window-scoped override only, the theme/city derives and every engine face
// stay byte-identical, outside the window the classic front resets exactly,
// and the window recurs every year (same day, all ends, one skin).
using Sy.Platform;
namespace Sy.MobTide
{
    /// &lt;summary&gt;One day's front: the garrison theme label and the day accent.
    /// Pure data, no engine types.&lt;/summary&gt;
    public sealed class CityVariant
    {
        public int index;          // 0..6 - the day's slot in the theme cycle
        public string label = "";  // garrison theme (banner + telemetry face)
        public int accentHex;      // day accent (the view converts to Color)
    }
    public static class CityRotation
    {
        public const int CycleDays = 7;
        public const int CityCount = 30;   // SaveWire unlocked-city mask cap (RotationCheck cross-asserts the two never
        // Embedded 7-day garrison themes: each variant carries its own banner
        // label and day accent - seven distinct faces, the week never repeats
        // a day and the front banner keeps the whole campaign map alive.
        static readonly int[] Accents = { 0xF2C14E, 0xE8734A, 0x9FD8CB, 0xB9A7E6, 0x8FBF6B, 0xF49FB4, 0x6C8EBF };
        static readonly string[] Themes = { "IRON WATCH", "RIVER SENTINELS", "STONE MILITIA", "MIDNIGHT GUARDS", "BRASS
        /// &lt;summary&gt;Day seed off the shared day key ("yyyyMMdd"): exact 8-digit
        /// parse; anything malformed degrades to seed 0 (local degradation -
        /// deterministic, never throws).&lt;/summary&gt;
        public static int DaySeed(string dayKey)
```

```
{
    if (dayKey == null || dayKey.Length != 8) return 0;
    int seed = 0;
    for (int i = 0; i &lt; 8; i++)
    {
        char c = dayKey[i];
        if (c &lt; '0' || c &gt; '9') return 0;
        seed = seed * 10 + (c - '0');
    }
    return seed;
}
/// &lt;summary&gt;Theme for a raw day seed (the single derive source: the
/// string face delegates here). Negative seeds fold back in-range
/// deterministically.&lt;/summary&gt;
public static CityVariant VariantForSeed(int seed)
{
    int i = seed % CycleDays;
    if (i &lt; 0) i += CycleDays;
    return new CityVariant
    {
        index = i,
        label = Themes[i],
        accentHex = Accents[i],
    };
}
/// &lt;summary&gt;Featured front city for a raw day seed: seed % CityCount,
/// negatives fold - 30 consecutive days tour every city exactly once.&lt;/summary&gt;
public static int CityForSeed(int seed)
{
    int i = seed % CityCount;
    if (i &lt; 0) i += CityCount;
    return i;
}
/// &lt;summary&gt;Theme for the shared day key (one boundary: the ad-cap day
/// bucket and the city front flip on the same local midnight).&lt;/summary&gt;
public static CityVariant VariantFor(string dayKey) =&gt; VariantForSeed(DaySeed(dayKey));
/// &lt;summary&gt;Featured front city for the shared day key (same day,
/// all ends, one city).&lt;/summary&gt;
public static int CityFor(string dayKey) =&gt; CityForSeed(DaySeed(dayKey));
/// &lt;summary&gt;Today's garrison theme (device-local clock - zero server faces).&lt;/summary&gt;
public static CityVariant Today() =&gt; VariantFor(AdCapPolicy.TodayKey());
/// &lt;summary&gt;Today's featured front city.&lt;/summary&gt;
public static int TodayCity() =&gt; CityFor(AdCapPolicy.TodayKey());
// ---- Festival window (G07-T032): the HOLIDAY TIDE skin ----
// Local calendar window 10-01..10-08, recurring every year: the
// holiday travel crowd mirrors into the front line while every other
// face - theme derive, city tour, engine - stays byte-identical.
public const int FestivalMonth = 10;
public const int FestivalFirstDay = 1;
public const int FestivalLastDay = 8;
```

```
        public const int FestivalAccentHex = 0xD7263D;   // festive crimson, distinct from all seven day accents
        /// &lt;summary&gt;Festival window off a raw day seed: month 10, days 1-8
        /// (calendar-recurring; malformed seeds fold outside the window).&lt;/summary&gt;
        public static bool IsFestivalSeed(int seed)
        {
            int month = (seed / 100) % 100;
            int day = seed % 100;
            return month == FestivalMonth &amp;amp;&amp;amp; day &gt;= FestivalFirstDay &amp;amp;&amp;amp; day &lt;= FestivalLastDay;
        }
        /// &lt;summary&gt;Festival window for a day key (same derive, single source).&lt;/summary&gt;
        public static bool IsFestival(string dayKey) =&gt; IsFestivalSeed(DaySeed(dayKey));
        /// &lt;summary&gt;Front banner line - the day's LiveOps face (banner + telemetry).
        /// Inside the festival window the line wears the HOLIDAY TIDE skin: day
        /// one brags the rail first-day record, the rest of the window the
        /// holiday gate line; outside the window the classic front returns
        /// byte-identical (the override is strictly window-scoped).&lt;/summary&gt;
        public static string BannerText(string dayKey)
        {
            int seed = DaySeed(dayKey);
            if (IsFestivalSeed(seed))
                return seed % 100 == FestivalFirstDay
                    ? "RAIL DAY ONE: 24.8M ON THE MOVE!"
                    : "HOLIDAY TIDE AT THE GATE!";
            return "FRONT CITY " + (CityForSeed(seed) + 1) + " - " + VariantForSeed(seed).label + " HOLDS TODAY";
        }
        /// &lt;summary&gt;Banner accent - the single accent source for the front
        /// line: festival crimson inside the window, the day's garrison accent
        /// outside (view and rigs read the same derive, one face).&lt;/summary&gt;
        public static int BannerAccentHex(string dayKey)
            =&gt; IsFestival(dayKey) ? FestivalAccentHex : VariantFor(dayKey).accentHex;
        /// &lt;summary&gt;Canonical signature - the dual-derive byte-parity
        /// instrument: two derives of one day that disagree anywhere
        /// disagree here (variant face + featured city face).&lt;/summary&gt;
        public static string Signature(CityVariant v, int city)
        {
            if (v == null) return "null";
            return "v" + v.index + '|' + v.label + "|a:" + v.accentHex.ToString("X6") + "|c:" + city;
        }
    }
}
// DailyChallenge.cs
// DailyChallenge.cs - the day's same-question gauntlet (G07-T016, G05 T017 law).
// Every day the ladder hands every player the same seed-derived wave span:
// the day seed the ad cap, the save and the city rotation already flip on
// (the T015 one-boundary law) picks a fixed 10-wave gauntlet inside the
// 200-wave ladder, so the whole world faces the identical question on the
// same local day - a system-preset daily rotation in pure offline form
// (X032: no server ever ships the question of the day). Unlike the G05
// privilege rod the gauntlet keeps the skill face: the player authors one
// route decision per wave, the run settles locally through the T013 codec
```

```
// pipeline (RunReplay = local recompute, zero RNG), and the result folds
// into a same-challenge code any device decodes and re-simulates
// byte-identically (share/import rivalry, red line 12). Daily-once
// accounting rides the T012 save wire: one day-key marker plus the day's
// settled captures, freed by the next day key through string compare alone.
using System.Collections.Generic;
using Sy.Platform;
namespace Sy.MobTide
{
    /// &lt;summary&gt;The day's question: the gauntlet span every player faces.&lt;/summary&gt;
    public sealed class DailySpan
    {
        public int seed;         // the shared day seed (T015 boundary)
        public int firstWave;    // gauntlet start inside the 200-wave ladder
        public int waveCount;    // gauntlet length (SpanWaves)
    }
    /// &lt;summary&gt;One settled daily gauntlet: the outcome faces plus the
    /// same-challenge code that reproduces the whole run on any device.&lt;/summary&gt;
    public sealed class DailyResult
    {
        public DailySpan span;
        public TideResolver resolver;   // the settled run (event ledger)
        public string code = "";        // same-challenge code (T013 codec)
        public int captures;
        public int losses;
        public int rescues;
        public long fold;
    }
    public static class DailyChallenge
    {
        public const int SpanWaves = 10;   // one gauntlet = ten waves
        /// &lt;summary&gt;Last legal gauntlet start (firstWave + SpanWaves must stay inside the ladder).&lt;/summary&gt
        public const int MaxSpanStart = FormationCodec.MaxWaves - SpanWaves;   // 190
        /// &lt;summary&gt;evt_daily_run funnel (T016): settle beats land here for
        /// gate assertions; S2 promotes this face to the unified pipeline.&lt;/summary&gt;
        public static readonly List&lt;string&gt; RunFunnel = new List&lt;string&gt;();
        /// &lt;summary&gt;Test hygiene: clears the funnel.&lt;/summary&gt;
        public static void Reset() =&gt; RunFunnel.Clear();
        /// &lt;summary&gt;Gauntlet start for a raw day seed: seed % legal-starts,
        /// negatives fold - every legal start is reachable, spans never
        /// leave the ladder.&lt;/summary&gt;
        public static int SpanStartForSeed(int seed)
        {
            int m = FormationCodec.MaxWaves - SpanWaves + 1;   // 191 legal starts
            int i = seed % m;
            if (i &lt; 0) i += m;
            return i;
        }
        /// &lt;summary&gt;The day's gauntlet span (malformed keys degrade to the
        /// seed-0 span, never throw - the T015 degradation law).&lt;/summary&gt;
```

```
public static DailySpan SpanFor(string dayKey)
{
    int seed = CityRotation.DaySeed(dayKey);
    return new DailySpan
    {
        seed = seed,
        firstWave = SpanStartForSeed(seed),
        waveCount = SpanWaves,
    };
}
/// &lt;summary&gt;Today's gauntlet (device-local clock - zero server faces).&lt;/summary&gt;
public static DailySpan TodaySpan() =&gt; SpanFor(AdCapPolicy.TodayKey());
/// &lt;summary&gt;Canonical signature - the dual-derive byte-parity
/// instrument: two derives of one day agree here or nowhere.&lt;/summary&gt;
public static string Signature(DailySpan s)
{
    if (s == null) return "null";
    return "d" + s.seed + "|w" + s.firstWave + "+" + s.waveCount;
}
/// &lt;summary&gt;Settles the day's gauntlet: one authored route per span
/// wave through the T013 replay pipeline (the codec's own local
/// recompute), the settled faces plus the shareable code. Route
/// arrays of the wrong length are a machine-readable fail (null,
/// no settle, no funnel beat - the G04 guard law, never a throw).
/// Rescue takes clamp into the codec field so no honest input can
/// produce a null code.&lt;/summary&gt;
public static DailyResult Play(string dayKey, RouteToken[] routes, int rescues)
{
    DailySpan span = SpanFor(dayKey);
    if (routes == null || routes.Length != span.waveCount) return null;
    if (rescues &lt; 0) rescues = 0;
    if (rescues &gt; FormationCodec.MaxRescues) rescues = FormationCodec.MaxRescues;
    var run = new FormationRun
    {
        firstWave = span.firstWave,
        routes = routes,
        rescues = rescues,
    };
    TideResolver r = FormationCodec.RunReplay(run);
    string code = FormationCodec.Encode(run);
    var res = new DailyResult
    {
        span = span,
        resolver = r,
        code = code == null ? "" : code,
        captures = r.Captures,
        losses = r.Losses,
        rescues = r.Rescues,
        fold = r.Population.Value,
    };
```

```
            RunFunnel.Add("evt_daily_run@" + dayKey
                + " span=W" + span.firstWave + "+W" + span.waveCount
                + " captures=" + res.captures
                + " losses=" + res.losses
                + " rescues=" + res.rescues
                + " fold=" + res.fold
                + " code=" + (res.code.Length == 0 ? "-" : res.code.Length + "ch"));
            return res;
        }
        /// &lt;summary&gt;Same-question gate: a code answers today's question only
        /// when its decoded span equals the day's span - a yesterday code or
        /// a custom-span code never poses as today's answer (malformed codes
        /// never throw, they simply do not pass).&lt;/summary&gt;
        public static bool IsDailyCode(string code, string dayKey)
        {
            FormationRun run = FormationCodec.Decode(code);
            if (run == null || run.routes == null) return false;
            DailySpan span = SpanFor(dayKey);
            return run.firstWave == span.firstWave &amp;&amp; run.routes.Length == span.waveCount;
        }
        /// &lt;summary&gt;Import face: decode + local recompute of any formation
        /// code (the T013 pipeline; null on malformed input). The daily
        /// wrapper adds no logic - the import face reuses the proven codec.&lt;/summary&gt;
        public static TideResolver ImportReplay(string code)
            =&gt; FormationCodec.RunReplay(FormationCodec.Decode(code));
        // ---- daily-once accounting (rides the T012 save wire) ----
        /// &lt;summary&gt;The day's gauntlet is settled when the save's marker
        /// equals the shared day key (string compare - the next local
        /// midnight frees the challenge by itself, no reset beat needed).&lt;/summary&gt;
        public static bool UsedToday() =&gt; SaveWire.DailyUsedToday();
        /// &lt;summary&gt;Captures the day's settled gauntlet booked (-1 = none).&lt;/summary&gt;
        public static int TodayCaptures() =&gt; SaveWire.DailyCapturesToday();
        /// &lt;summary&gt;Books the settled gauntlet: one day-key marker plus the
        /// day's captures, persisted on the booking beat itself; the first
        /// result of the day stands (a second settle same day books nothing).&lt;/summary&gt;
        public static void MarkUsed(DailyResult r)
        {
            SaveWire.OnDailyChallenge(r == null ? -1 : r.captures);
        }
    }
}
// DoublerGate.cs
// DoublerGate.cs - the tide-double ad gate (G07-T014-b, the second IAA point).
// Manual section 6 verdict: candidate 1 "tide double" adopted (E3=8.8) - the
// doubler doubles the CURRENT wave's population fold through the single
// PopulationCounter source: a completed rewarded view on the capture-fold
// window injects the wave's unit count a second time (fold x2 for that wave,
// wave-bound, never cross-wave persistent). Pure additive seam - the resolver
// law is untouched (decide face deterministic, frozen constants intact), and
// a run that never taps books byte-identical events (bot replay zero drift,
```

```
// the S1 fold baseline stands). The fold window is derived from the
// resolver's public event stream (zero host wiring): it opens on
// gate_captured/rescued_capture and closes on the next wave_start - a wipe
// folds nothing, so a loss wave never arms the doubler. Category chain
// mirrors RescueGate (T009): HasCategoryBudget precheck and ShowRewarded
// consume ride the same policy instance and day bucket
// (g07_tide_double -&gt; shared double 5/day), the evt_ad funnel books
// offer -&gt; show -&gt; complete -&gt; double_landed in order. Rollback = delete
// this file (plus the check family).
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.MobTide
{
    /// &lt;summary&gt;The tide-double gate: books offers on the capture fold, gates
    /// the CTA tap through the double ad category chain, and lands the x2
    /// fold injection when the rewarded view completes.&lt;/summary&gt;
    public sealed class DoublerGate
    {
        /// &lt;summary&gt;Tide-double point id (T014-b): draws from the shared
        /// double 5/day category - the same budget every settle/pay doubler
        /// draws (AdCategories.CategoryOfPoint authority).&lt;/summary&gt;
        public const string DoublePoint = "g07_tide_double";
        readonly TideResolver resolver;
        readonly AdCapPolicy adCap;
        readonly IAdsService ads;
        readonly Func&lt;string&gt; dayKey;
        public readonly List&lt;string&gt; AdFunnel = new List&lt;string&gt;();
        int _doubledWaveIndex = -1;   // the wave already doubled (x2 lands once per wave)
        public DoublerGate(TideResolver resolver, AdCapPolicy adCap, IAdsService ads, Func&lt;string&gt; dayKey = null)
        {
            this.resolver = resolver;
            this.adCap = adCap;
            this.ads = ads;
            // the precheck reads the very bucket the registered service consumes
            // (injected key only for deterministic gates; production rides TodayKey)
            this.dayKey = dayKey ?? AdCapPolicy.TodayKey;
            if (ads != null) ads.OnAdEvent += OnAdLifecycle;
        }
        /// &lt;summary&gt;Live double-category precheck: always-fresh budget face on
        /// the same policy instance and the same day bucket the registered
        /// service consumes from, so precheck and consume can never disagree.&lt;/summary&gt;
        public bool OfferBudgetOk
        {
            get
            {
                return adCap != null
                    &amp;&amp; adCap.HasCategoryBudget(dayKey(), AdCategories.CategoryOfPoint(DoublePoint));
            }
```

```
}
/// &lt;summary&gt;The capture-fold window: the last resolved wave folded its
/// people (direct capture or rescued comeback) and no new wave has
/// started - the only window a double tap may enter the ad chain.
/// Derived from the resolver's public event stream; once the wave is
/// doubled the window closes until the next wave folds.&lt;/summary&gt;
public bool DoubleArmed
{
    get { return WindowWaveIndex() &gt;= 0; }
}
string Stamp()
{
    return resolver == null ? "0.00" : resolver.SimTime.ToString("0.00");
}
/// &lt;summary&gt;Offer booking on the capture beat: every live fold books
/// its offer same-tick - capped offers book too, the honest face
/// refuses at the tap, never after the ad.&lt;/summary&gt;
public void NoteCaptureFold()
{
    if (!DoubleArmed) return;
    bool ok = OfferBudgetOk;
    AdFunnel.Add("evt_ad_offer" + (ok ? "" : "_capped") + "@" + Stamp());
}
/// &lt;summary&gt;CTA tap: armed guard - budget precheck - rewarded view -
/// x2 fold injection. The ad only ever plays on an open fold window
/// with budget left; the injection lands only when the view completed
/// and lands at most once per wave.&lt;/summary&gt;
public async Task&lt;bool&gt; AcceptFromCta()
{
    if (resolver == null || ads == null) return false;
    int waveIdx = WindowWaveIndex();          // captured at entry - the wave the player doubles
    if (waveIdx &lt; 0) return false;            // dead tap: no offer, no ad, no burn
    if (!OfferBudgetOk)
    {
        AdFunnel.Add("evt_ad_offer_capped@" + Stamp());
        return false;
    }
    RewardResult r = await ads.ShowRewarded(DoublePoint);
    if (r == null || !r.Completed) return false;   // evt_ad_fail rides the lifecycle face; window stays open fo
    // the x2 injection: the wave's fold + this injection = fold x2 for
    // that wave, through the single PopulationCounter source.
    resolver.Population.AddUnits(WaveTable.SpecFor(waveIdx).units);
    _doubledWaveIndex = waveIdx;
    AdFunnel.Add("evt_double_landed@" + Stamp());
    return true;
}
/// &lt;summary&gt;The open fold window's wave index, -1 when closed: the
/// last wave_start with a fold event after it (gate_captured or
/// rescued_capture) and no newer wave_start. A loss with no rescue
/// books no fold - nothing to double.&lt;/summary&gt;
```

```
        int WindowWaveIndex()
        {
            if (resolver == null) return -1;
            int lastStart = -1;
            for (int i = resolver.Events.Count - 1; i &gt;= 0; i--)
                if (resolver.Events[i].type == "wave_start") { lastStart = i; break; }
            if (lastStart &lt; 0) return -1;
            int fold = -1, loss = -1;
            for (int i = lastStart + 1; i &lt; resolver.Events.Count; i++)
            {
                string t = resolver.Events[i].type;
                if (fold &lt; 0 &amp;&amp; (t == "gate_captured" || t == "rescued_capture")) fold = i;
                if (loss &lt; 0 &amp;&amp; t == "city_loss") loss = i;
            }
            if (fold &lt; 0) return -1;                 // nothing folded - nothing to double
            if (loss &gt; fold) return -1;              // unreachable (resolve is atomic) - kept for face safety
            int waveIdx = ParseWaveIndex(resolver.Events[lastStart].detail);
            if (waveIdx &lt; 0 || waveIdx == _doubledWaveIndex) return -1;
            return waveIdx;
        }
        /// &lt;summary&gt;Parses the wave index out of a wave_start detail
        /// ("wave 12 (Pincer) - split:3 meets guard 8" -&gt; 12).&lt;/summary&gt;
        static int ParseWaveIndex(string waveStartDetail)
        {
            if (string.IsNullOrEmpty(waveStartDetail)) return -1;
            int at = waveStartDetail.IndexOf("wave ");
            if (at &lt; 0) return -1;
            int p = at + 5, n = 0;
            bool any = false;
            while (p &lt; waveStartDetail.Length &amp;&amp; char.IsDigit(waveStartDetail[p]))
            {
                n = n * 10 + (waveStartDetail[p] - '0');
                p++;
                any = true;
            }
            return any ? n : -1;
        }
        /// &lt;summary&gt;evt_ad funnel (T014-b): lifecycle events of the
        /// tide-double point land in the funnel list for gate assertions; S2
        /// promotes this face to the unified telemetry pipeline.&lt;/summary&gt;
        void OnAdLifecycle(AdLifecycle e)
        {
            if (e.PointId != DoublePoint) return;
            AdFunnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + Stamp());
        }
    }
}
// FormationCodec.cs
// FormationCodec.cs - offline-versus formation codes for MobTide (G07-T013, X032 zero-server rivalry).
// A run is pure authored input: which route each wave took (single / split3 /
```

```
// ladder) plus how many comebacks the run booked. The resolver is zero-RNG
// end to end (the E4 law), so the whole authored layer folds into one
// paste-anywhere code: encode the run, ship it through any share channel,
// decode on a friend's device and the local machine re-simulates the
// identical event stream - no server, no matchmaking, no netcode (red line
// 12). The rescue policy inside a code is "the first R losses take the
// comeback" - the daily ad cap itself is a host concern (RescueGate), the
// code only carries the accounting (G05 ReplayCodec law, mirrored).
// Layout (bit stream, MSB first): v[4] firstWave[8] waveCount[8] rescues[4]
// routes[waveCount x 2] -&gt; pad-to-byte zeros -&gt; + checksum byte
// -&gt; "MT" + base62 body (canonical form, no leading zeros).
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.MobTide
{
    /// &lt;summary&gt;Route vocabulary of one authored wave decision (2 bits).&lt;/summary&gt;
    public enum RouteToken : byte
    {
        Single = 0,   // the whole tide through one lane (the plain answer)
        Split3 = 1,   // three columns at once - the pincer encirclement answer
        Ladder = 2    // stack the ladder - the only answer a wall ever takes
    }
    /// &lt;summary&gt;A shareable run descriptor - everything a local replay needs.&lt;/summary&gt;
    public sealed class FormationRun
    {
        public int v = 1;             // codec version (migration hook, same law as the save schema)
        public int firstWave;          // ladder span start (0..199)
        public int waveCount;          // authored waves in the span (1..200)
        public RouteToken[] routes;     // one decision per wave in the span
        public int rescues;            // comebacks taken at the first R losses
    }
    public static class FormationCodec
    {
        public const string Prefix = "MT";
        public const string Alphabet = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
        public const int CodeBudgetChars = 80;   // share-channel budget the codec must honor
        public const int MaxWaves = 200;         // the in-table ladder span (WaveTable law)
        public const int MaxRescues = 15;        // 4-bit rescue accounting field
        const int VersionBits = 4, FirstWaveBits = 8, WaveCountBits = 8, RescueBits = 4, RouteBits = 2;
        const int HeaderBits = VersionBits + FirstWaveBits + WaveCountBits + RescueBits;   // 24
        const int MaxPayloadBytes = (HeaderBits + MaxWaves * RouteBits + 7) / 8;           // 53
        const int MaxBodyChars = 73;   // 54 packed bytes max -&gt; 73 base62 digits
        /// &lt;summary&gt;Encodes a run into a shareable code ("MT" + base62).
        /// Host-side contract: the span must sit inside the 200-wave ladder,
        /// every token must be a real route verb, the rescue take must fit
        /// the field - violations return null (programming errors fail loud,
        /// G05 guard law).&lt;/summary&gt;
        public static string Encode(FormationRun run)
```

```
{
    if (run == null || run.routes == null) return null;
    int waveCount = run.routes.Length;
    if (waveCount &lt; 1 || waveCount &gt; MaxWaves) return null;
    if (run.firstWave &lt; WaveTable.FirstWave || run.firstWave + waveCount &gt; WaveTable.LastWave + 1) return
    if (run.rescues &lt; 0 || run.rescues &gt; MaxRescues) return null;
    var bits = new BitWriter();
    bits.Write((uint)run.v &amp; 0xFu, VersionBits);
    bits.Write((uint)run.firstWave, FirstWaveBits);
    bits.Write((uint)waveCount, WaveCountBits);
    bits.Write((uint)run.rescues, RescueBits);
    for (int i = 0; i &lt; waveCount; i++)
    {
        RouteToken t = run.routes[i];
        if (t != RouteToken.Single &amp;&amp; t != RouteToken.Split3 &amp;&amp; t != RouteToken.Ladder) return n
        bits.Write((uint)t, RouteBits);
    }
    byte[] payload = bits.ToArray();
    var pack = new byte[payload.Length + 1];
    Array.Copy(payload, pack, payload.Length);
    pack[payload.Length] = Checksum(payload);
    return Prefix + BytesToBody(pack);
}
/// &lt;summary&gt;Decodes a formation code. Returns null on any malformed
/// input: wrong prefix, illegal char, overflowing body, bad checksum,
/// unknown version, out-of-ladder span, dirty padding, token 3 -
/// a corrupted code never replays, never throws.&lt;/summary&gt;
public static FormationRun Decode(string code)
{
    if (code == null) return null;
    if (!code.StartsWith(Prefix, StringComparison.Ordinal)) return null;
    string body = code.Substring(Prefix.Length);
    if (body.Length == 0 || body.Length &gt; MaxBodyChars) return null;
    byte[] pack;
    if (!TryBodyToBytes(body, MaxPayloadBytes + 1, out pack)) return null;
    if (pack.Length &lt; (HeaderBits + 7) / 8 + 1) return null;
    int payloadLen = pack.Length - 1;
    var payload = new byte[payloadLen];
    Array.Copy(pack, payload, payloadLen);
    if (Checksum(payload) != pack[payloadLen]) return null;
    var bits = new BitReader(payload);
    uint v = bits.Read(VersionBits);
    if (v != 1u) return null;
    uint firstWave = bits.Read(FirstWaveBits);
    uint waveCount = bits.Read(WaveCountBits);
    uint rescues = bits.Read(RescueBits);
    if (waveCount &lt; 1u || waveCount &gt; MaxWaves) return null;
    if (firstWave + waveCount &gt; (uint)(WaveTable.LastWave + 1)) return null;
    if (payloadLen != (HeaderBits + (int)waveCount * RouteBits + 7) / 8) return null;   // canonical length only
    var run = new FormationRun();
```

```
        run.v = (int)v;
        run.firstWave = (int)firstWave;
        run.waveCount = (int)waveCount;
        run.rescues = (int)rescues;
        run.routes = new RouteToken[(int)waveCount];
        for (int i = 0; i &lt; run.routes.Length; i++)
        {
            uint t = bits.Read(RouteBits);
            if (t &gt; 2u) return null;   // token 3 does not exist
            run.routes[i] = (RouteToken)t;
        }
        if (!bits.RestAllZero()) return null;   // padding must stay clean (canonical form)
        return run;
    }
    /// &lt;summary&gt;Route verb -&gt; concrete plan for one wave (the same plans
    /// the production choreography uses - the import face reuses this).&lt;/summary&gt;
    public static RoutePlan PlanFor(RouteToken token, WaveSpec spec)
    {
        if (spec == null) throw new ArgumentNullException("spec");
        switch (token)
        {
            case RouteToken.Ladder:
                return new RoutePlan { laneUnits = new[] { spec.units }, useLadder = true };
            case RouteToken.Split3:
                return SplitValve.Split3Plan(spec.units);
            default:
                return SplitValve.SingleRoutePlan(spec.units);
        }
    }
    /// &lt;summary&gt;Local recompute: re-simulates the run from its descriptor
    /// with the frozen E4 cadence (the S1 bot ladder: distinct loss ticks,
    /// the rescue surge inside the span). Deterministic byte-for-byte on
    /// any device - the zero-server rivalry core. The routes array is the
    /// authoritative payload (waveCount is its shadow, always in step).&lt;/summary&gt;
    public static TideResolver RunReplay(FormationRun run)
    {
        var r = new TideResolver();
        if (run == null || run.routes == null) return r;
        int rescuesLeft = run.rescues;
        int waves = run.routes.Length;
        for (int i = 0; i &lt; waves; i++)
        {
            int w = run.firstWave + i;
            WaveSpec spec = WaveTable.SpecFor(w);
            r.SimTime = 1.00 + w * 0.60;
            AssaultOutcome outcome = r.Resolve(spec, PlanFor(run.routes[i], spec));
            if (outcome == AssaultOutcome.Lost &amp;&amp; rescuesLeft &gt; 0 &amp;&amp; r.BookRescue())
                rescuesLeft--;
        }
        return r;
```

```
}
/// &lt;summary&gt;Event-stream dump for parity proofs (checks, ghost compare faces).&lt;/summary&gt;
public static string DumpEvents(TideResolver r)
{
    var sb = new StringBuilder();
    if (r == null) return "";
    for (int i = 0; i &lt; r.Events.Count; i++)
    {
        ValveEvent e = r.Events[i];
        sb.Append(e.t.ToString("F2", CultureInfo.InvariantCulture))
          .Append('|').Append(e.type).Append('|').AppendLine(e.detail);
    }
    sb.Append("COUNTS ").Append(r.Resolves).Append('/').Append(r.Captures)
      .Append('/').Append(r.Losses).Append('/').Append(r.Rescues)
      .Append('/').Append(r.Population.Value);
    return sb.ToString();
}
// ---- packing plumbing (MSB-first bit stream, big-endian value) ----
static byte Checksum(byte[] payload)
{
    uint c = 0x5Bu;
    for (int i = 0; i &lt; payload.Length; i++) c = ((c ^ payload[i]) * 0x5Du) &amp; 0xFFu;
    return (byte)c;
}
static string BytesToBody(byte[] bytes)
{
    var v = (byte[])bytes.Clone();
    var sb = new StringBuilder();
    int len = v.Length;
    while (len &gt; 0)
    {
        uint carry = 0;
        bool any = false;
        for (int i = 0; i &lt; len; i++)
        {
            uint cur = (carry &lt;&lt; 8) | v[i];
            v[i] = (byte)(cur / 62u);
            if (v[i] != 0) any = true;
            carry = cur % 62u;
        }
        sb.Append(Alphabet[(int)carry]);
        if (!any) break;
        int first = 0;
        while (first &lt; len &amp;&amp; v[first] == 0) first++;
        if (first &gt; 0) { Array.Copy(v, first, v, 0, len - first); len -= first; }
    }
    // digits were appended least-significant first; decode reads body[0] as the leading digit
    int a = 0, b = sb.Length - 1;
    while (a &lt; b) { char t = sb[a]; sb[a] = sb[b]; sb[b] = t; a++; b--; }
    return sb.ToString();
```

```
}
static bool TryBodyToBytes(string s, int maxBytes, out byte[] bytes)
{
    bytes = null;
    var val = new byte[maxBytes];
    int len = 0;
    for (int i = 0; i &lt; s.Length; i++)
    {
        int d = Alphabet.IndexOf(s[i]);
        if (d &lt; 0) return false;
        uint carry = (uint)d;
        for (int j = len - 1; j &gt;= 0; j--)
        {
            uint cur = val[j] * 62u + carry;
            val[j] = (byte)(cur &amp; 0xFFu);
            carry = cur &gt;&gt; 8;
        }
        while (carry &gt; 0)
        {
            if (len &gt;= maxBytes) return false;   // overflow = malformed
            Array.Copy(val, 0, val, 1, len);
            val[0] = (byte)(carry &amp; 0xFFu);
            carry &gt;&gt;= 8;
            len++;
        }
    }
    bytes = new byte[len];
    Array.Copy(val, bytes, len);
    return true;
}
sealed class BitWriter
{
    readonly List&lt;byte&gt; _bytes = new List&lt;byte&gt;();
    int _bit;   // bits filled in the last byte (0..7)
    public void Write(uint value, int count)
    {
        for (int i = count - 1; i &gt;= 0; i--)
        {
            if (_bit == 0) _bytes.Add(0);
            if ((value &amp; (1u &lt;&lt; i)) != 0u)
                _bytes[_bytes.Count - 1] |= (byte)(1u &lt;&lt; (7 - _bit));
            _bit = (_bit + 1) &amp; 7;
        }
    }
    public byte[] ToArray() { return _bytes.ToArray(); }
}
sealed class BitReader
{
    readonly byte[] _bytes;
    int _pos;   // absolute bit position
```

```
            public BitReader(byte[] bytes) { _bytes = bytes; }
            public uint Read(int count)
            {
                uint v = 0;
                for (int i = 0; i &lt; count; i++)
                {
                    int idx = _pos &gt;&gt; 3;
                    if (idx &gt;= _bytes.Length) { _pos = _bytes.Length * 8; return uint.MaxValue; }   // underrun = cal
                    v = (v &lt;&lt; 1) | (uint)((_bytes[idx] &gt;&gt; (7 - (_pos &amp; 7))) &amp; 1);
                    _pos++;
                }
                return v;
            }
            public bool RestAllZero()
            {
                for (int p = _pos; p &lt; _bytes.Length * 8; p++)
                    if (((_bytes[p &gt;&gt; 3] &gt;&gt; (7 - (p &amp; 7))) &amp; 1) != 0) return false;
                return true;
            }
        }
    }
}
// LadderSiege.cs
// LadderSiege.cs - wall-wave human ladder (G07-T007, E4 graybox wall model migrated).
// A wall wave (guard 999) never falls to a frontal push - the only way through is
// over: the tide locks arms into a three-layer human ladder inside the 1.50s stack
// window, pours over the top and takes the gate from behind. The forced teach rides
// the first wall (E4 wave2 = index 1): the prompt drops with the wall face, the
// scripted hold starts 2.00s later and the first ladder stands 3.50s after the
// prompt (E4 Stage1 measured, red-line cap 60s). Counters keep the E4 names so the
// check re-runs the graybox wall numbers exactly (ladder 25 wins / brute 0 wins
// over the 50 wall waves of the 200-wave ladder).
// Pure C# with a settable logical clock - headless gates drive it deterministically.
using System;
using System.Collections.Generic;
namespace Sy.MobTide
{
    public sealed class LadderSiege
    {
        // E4 frozen wall timeline (only-up law: these may only ever tighten).
        public const double StackWindow = 1.50;        // three layers lock inside this window
        public const int LockArmLayers = 3;            // one layer locks per window slice
        public const double TeachFirstUseAt = 3.50;   // E4 Stage1: first ladder stands this long after the prompt
        public const double TeachFirstUseCap = 60.0;  // red-line-11 cap on the first use
        public const double TeachHoldStartAt = TeachFirstUseAt - StackWindow; // scripted reaction beat
        public double SimTime;
        public readonly List&lt;ValveEvent&gt; Events = new List&lt;ValveEvent&gt;();
        // In-run E4 counters (Stage2 names kept for the parity re-run).
        public int LadderAttempts, LadderCaptures;
        public int BruteAttempts, BruteLosses;
```

```
bool _teachActive, _firstLadderUsed;
public void Log(string type, string detail)
{
    Events.Add(new ValveEvent { t = SimTime, type = type, detail = detail });
}
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
/// &lt;summary&gt;Hold-face helper: how many lock-arm layers the held time has
/// locked (one per window slice, capped at the full ladder). The view lights
/// its layer blocks off this, the gate freezes the cadence.&lt;/summary&gt;
public static int LayersForHold(double held)
{
    if (held &lt;= 0.0) return 0;
    double slice = StackWindow / LockArmLayers;
    int layers = (int)Math.Floor(held / slice);
    return layers &gt; LockArmLayers ? LockArmLayers : layers;
}
/// &lt;summary&gt;Forced teach on the first wall: the prompt drops with the wall
/// face and the first ladder use answers with evt_ladder_build.&lt;/summary&gt;
public void BeginTeach()
{
    _teachActive = true;
    Log("ladder_teach", "HOLD to stack a human ladder - over the wall");
}
/// &lt;summary&gt;One wall-wave assault (E4 frozen model). Ladder plan: the stack
/// spends the window, then the climb beats land together - over the top, the
/// gate falls from behind (the frontal guard line never applies to a climb).
/// Brute plan: units vs guard 999 - certain loss at table sizes, rescue
/// offered on the wipe. Programming errors (null spec/plan, wrong lane
/// count, non-wall spec) fail loud - this core only ever answers the wall.&lt;/summary&gt;
public AssaultOutcome Assault(WaveSpec spec, RoutePlan plan)
{
    if (spec == null) throw new ArgumentNullException("spec");
    if (plan == null || plan.laneUnits == null || plan.laneUnits.Length &lt; 1 || plan.laneUnits.Length &gt; 3)
        throw new ArgumentException("route plan needs 1..3 lanes");
    if (spec.type != WaveType.Wall) throw new ArgumentException("LadderSiege resolves wall waves only");
    double start = SimTime;
    if (plan.useLadder)
    {
        // Decision delegated to the SYS04 authority (G07-T008): over the
        // top always takes the gate. If this ever disagrees the combat
```

```
                // law has drifted - fail loud, never silently mis-choreograph.
                if (TideResolver.Decide(spec, plan).outcome != AssaultOutcome.Captured)
                    throw new InvalidOperationException("wall law broken: ladder must capture");
                LadderAttempts++;
                Log("stack_hold", "the tide locks arms - window " + StackWindow.ToString("0.00") + "s");
                SimTime = start + StackWindow;
                Log("ladder_build", "human ladder stands - " + LockArmLayers + " layers lock arms");
                if (_teachActive &amp;&amp; !_firstLadderUsed)
                {
                    Log("first_ladder_use", "teach answered - first ladder stands");
                    _firstLadderUsed = true;
                }
                Log("wall_climb", "the tide pours over the top");
                Log("gate_captured", "the gate falls from behind");
                Log("gold_rain", "coins rain down");
                LadderCaptures++;
                return AssaultOutcome.Captured;
            }
            // Decision delegated to the SYS04 authority (G07-T008): guard 999
            // means a frontal push never wins at any table size.
            if (TideResolver.Decide(spec, plan).outcome != AssaultOutcome.Lost)
                throw new InvalidOperationException("wall law broken: brute never wins");
            BruteAttempts++;
            Log("brute_push", "whole tide pushes the wall head-on");
            SimTime = start + StackWindow; // the push spends the window, the wall holds
            Log("city_loss", "the wall holds - the wave is wiped out");
            Log("rescue_offer", "Reverse Redemption? (rewarded-ad fail-retry slot)");
            BruteLosses++;
            return AssaultOutcome.Lost;
        }
    }
}
// PopulationCounter.cs
// PopulationCounter.cs - the tide's running headcount (G07-T004).
// Every captured gate folds its whole attacking wave into the population; the
// counter is the brag face of the share card (T010: "99,999 people took this
// gate"). The display cap keeps the overflow honest: past 99,999 the card keeps
// reading 99,999 instead of rolling into a smaller-looking number.
using System.Globalization;
namespace Sy.MobTide
{
    public sealed class PopulationCounter
    {
        public const long DisplayCap = 99999;
        public long Value { get; private set; }
        /// &lt;summary&gt;A gate fell: its whole wave joins the population.&lt;/summary&gt;
        public void AddUnits(int units)
        {
            if (units &gt; 0) Value += units;
        }
```

```
        /// &lt;summary&gt;Raw headcount, comma-grouped (telemetry face).&lt;/summary&gt;
        public string RawValue()
            =&gt; Value.ToString("N0", CultureInfo.InvariantCulture);
        /// &lt;summary&gt;Card face: capped at 99,999 so the brag never shrinks.&lt;/summary&gt;
        public string DisplayValue()
            =&gt; Value &gt;= DisplayCap ? DisplayCap.ToString("N0", CultureInfo.InvariantCulture)
                                : RawValue();
        public void Reset() =&gt; Value = 0;
    }
}
// RescueGate.cs
// RescueGate.cs - the ad-gated "Reverse Redemption" interaction face (G07-T009).
// The city-loss fail-retry slot: every booked loss books its offer on the very
// loss tick (the run loop calls NoteCityLoss right after a Lost resolve), and
// accepting runs the rewarded-ad category chain before the resolver books the
// population counter-attack - HasCategoryBudget precheck on the shared
// AdCapPolicy, ShowRewarded on the registered service (category-capped
// consumption on the same policy instance), the evt_ad funnel face for the S2
// telemetry promotion. Pure logic, no MonoBehaviour: the scene host wires CTA
// taps to AcceptFromCta, headless gates drive it directly. Zero RNG holds -
// the comeback never re-rolls the combat, the resolver books the scripted
// counter-attack capture (E4 Rescue law).
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.MobTide
{
    /// &lt;summary&gt;The rescue gate: books offers on the city-loss tick, gates the
    /// CTA tap through the ad category chain, and lands the counter-attack
    /// when the rewarded view completes.&lt;/summary&gt;
    public sealed class RescueGate
    {
        /// &lt;summary&gt;Rescue point id (T009): draws from the shared rescue 4/day
        /// category - the fail-retry slot exists in every loop, never dominates
        /// it (E3#4: rescue 1-5 per 200, offer always same-tick).&lt;/summary&gt;
        public const string RescuePoint = "g07_rescue";
        readonly TideResolver resolver;
        readonly AdCapPolicy adCap;
        readonly IAdsService ads;
        readonly Func&lt;string&gt; dayKey;
        public readonly List&lt;string&gt; AdFunnel = new List&lt;string&gt;();
        public RescueGate(TideResolver resolver, AdCapPolicy adCap, IAdsService ads, Func&lt;string&gt; dayKey = null)
        {
            this.resolver = resolver;
            this.adCap = adCap;
            this.ads = ads;
            // the precheck reads the very bucket the registered service consumes
            // (injected key only for deterministic gates; production rides TodayKey)
            this.dayKey = dayKey ?? AdCapPolicy.TodayKey;
```

```
    if (ads != null) ads.OnAdEvent += OnAdLifecycle;
}
/// &lt;summary&gt;Live rescue category precheck: always-fresh budget face on
/// the same policy instance and the same day bucket the registered
/// service consumes from, so precheck and consume can never disagree.&lt;/summary&gt;
public bool OfferBudgetOk
{
    get
    {
        return adCap != null
            &amp;&amp; adCap.HasCategoryBudget(dayKey(), AdCategories.CategoryOfPoint(RescuePoint));
    }
}
/// &lt;summary&gt;True while the fail-retry slot is live: a city loss just
/// resolved and no next wave has started - the only window a rescue
/// tap may even enter the ad chain.&lt;/summary&gt;
public bool RescueArmed
{
    get { return resolver != null &amp;&amp; resolver.RescueArmed; }
}
string Stamp()
{
    return resolver == null ? "0.00" : resolver.SimTime.ToString("0.00");
}
/// &lt;summary&gt;Offer booking on the city-loss tick: every live loss books
/// its offer same-tick - capped offers book too, the honest face
/// refuses at the tap, never after the ad.&lt;/summary&gt;
public void NoteCityLoss()
{
    if (!RescueArmed) return;
    bool ok = OfferBudgetOk;
    AdFunnel.Add("evt_ad_offer" + (ok ? "" : "_capped") + "@" + Stamp());
}
/// &lt;summary&gt;CTA tap: armed guard - budget precheck - rewarded view -
/// counter-attack booking. The ad only ever plays on an armed loss with
/// budget left; the resolver books the rescue only when the view
/// completed and the comeback chain lands on the surge beat.&lt;/summary&gt;
public async Task&lt;bool&gt; AcceptFromCta()
{
    if (resolver == null || ads == null) return false;
    if (!RescueArmed) return false;
    if (!OfferBudgetOk)
    {
        AdFunnel.Add("evt_ad_offer_capped@" + Stamp());
        return false;
    }
    RewardResult r = await ads.ShowRewarded(RescuePoint);
    if (r != null &amp;&amp; r.Completed &amp;&amp; resolver.BookRescue())
    {
        // rescue_landed books on the surge beat - the resolver already
```

```
                // advanced SimTime past the counter-attack surge when the
                // rescued capture landed.
                AdFunnel.Add("evt_rescue_landed@" + Stamp());
                return true;
            }
            return false;
        }
        /// &lt;summary&gt;evt_ad funnel (T009): lifecycle events of the rescue point
        /// land in the funnel list for gate assertions; S2 promotes this face
        /// to the unified telemetry pipeline.&lt;/summary&gt;
        void OnAdLifecycle(AdLifecycle e)
        {
            if (e.PointId != RescuePoint) return;
            AdFunnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + Stamp());
        }
    }
}
// SaveWire.cs
// SaveWire.cs - local-authority save model for MobTide meta progression (G07-T012).
// One flat model carries the four persisted faces the S2 loop needs from day
// one: coins, the banked lifetime population (the brag number every share card
// rides on), the wave progress index, and the unlocked-city bitmap. Local
// storage is the authority - this wire never calls CloudSync (red line 12,
// zero-server); the Resolver seam lets SaveFlowCheck prove that doctrine with
// a counting spy instead of trusting the source shape. The ActiveSlot seam
// keeps batch gates hermetic: the host path reads/writes the main slot, checks
// point it at a probe slot.
// Host contract: TideStage.CaptureBooker wires OnGateCaptured on the capture
// beat (the gate falls -&gt; its whole wave joins the banked population, rains
// its gold, advances the progress index and unlocks its city), so the meta
// face persists the instant the beat lands, never on a timer.
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.MobTide
{
    /// &lt;summary&gt;Persisted meta progression (flat model, MiniJson round-trip).&lt;/summary&gt;
    public sealed class G07SaveModel
    {
        public int v = 1;          // schema version (migration hook)
        public int coins;          // banked gold from captured gates
        public long population;    // banked lifetime headcount (share-card brag face)
        public int waveIndex;      // progress: first unresolved wave (0-based)
        public int citiesMask;     // unlocked-city bitmap, bit i = city i
        public string dailyKey = "";      // day key whose gauntlet is settled (T016)
        public int dailyCaptures = -1;     // that day's settled captures (-1 = none)
    }
    public static class SaveWire
    {
```

```
public const string MainSlot = "g07_main";
public const int CoinPerUnit = 10;   // placeholder economy: a captured gate rains unit gold (city loop tunes it
/// &lt;summary&gt;Slot the host path reads/writes; checks point it at a probe
/// slot so batch runs stay hermetic against the real save.&lt;/summary&gt;
public static string ActiveSlot = MainSlot;
/// &lt;summary&gt;Service seam: default resolves the registered ISaveService;
/// SaveFlowCheck swaps in a CloudSync-counting spy (a doctrine needs an
/// instrument).&lt;/summary&gt;
public static Func&lt;ISaveService&gt; Resolver = ResolveRegistered;
/// &lt;summary&gt;In-memory authority: loaded on boot, persisted on every beat worth saving.&lt;/summary&gt;
public static readonly G07SaveModel State = new G07SaveModel();
public static void LoadOrDefault() { LoadOrDefault(ActiveSlot); }
public static void Persist() { Persist(ActiveSlot); }
/// &lt;summary&gt;Boot read: missing or corrupt payload =&gt; defaults (fresh
/// start: city 0 unlocked, zero banks) - a crashed save never bricks
/// the run, it restarts the meta face honestly.&lt;/summary&gt;
public static void LoadOrDefault(string slot)
{
    ISaveService save = Resolver();
    string json = save == null ? null : save.Load(slot);
    G07SaveModel m = string.IsNullOrEmpty(json) ? null : Deserialize(json);
    CopyInto(m ?? DefaultModel(), State);
}
public static void Persist(string slot)
{
    ISaveService save = Resolver();
    if (save != null) save.Save(slot, Serialize(State));
}
/// &lt;summary&gt;Fresh-install / crash-recovery model: the opening city is
/// playable, everything else reads zero.&lt;/summary&gt;
public static G07SaveModel DefaultModel()
{
    var m = new G07SaveModel();
    m.citiesMask = 1 &lt;&lt; 0;
    return m;
}
/// &lt;summary&gt;Capture beat (host contract, wired to
/// TideStage.CaptureBooker): the gate folds its wave into the banked
/// population, rains the gold, advances the progress index (monotonic
/// - a replayed wave never regresses progress) and unlocks the
/// opening city the wave belongs to, then persists on the beat
/// itself.&lt;/summary&gt;
public static void OnGateCaptured(WaveSpec w)
{
    OnGateCapturedInCity(w, 0);
}
/// &lt;summary&gt;City-qualified capture face (S2 city loop entry): same
/// booking law, the unlocked bit lands on the city the wave's gate
/// belongs to.&lt;/summary&gt;
public static void OnGateCapturedInCity(WaveSpec w, int cityIndex)
```

```
{
    if (w == null) return;
    State.population += w.units &gt; 0 ? w.units : 0;
    State.coins += CoinsForGate(w.units);
    State.waveIndex = Math.Max(State.waveIndex, w.waveIndex + 1);
    UnlockCity(cityIndex);
}
/// &lt;summary&gt;City unlock booking (S2 city loop entry): sets the bit and
/// persists - unlocks are permanent meta progression.&lt;/summary&gt;
public static void UnlockCity(int cityIndex)
{
    if (cityIndex &lt; 0 || cityIndex &gt; 29) return;   // bitmap face: 30 cities max, out-of-range ignores
    State.citiesMask |= 1 &lt;&lt; cityIndex;
    Persist();
}
public static bool IsCityUnlocked(int cityIndex)
    =&gt; cityIndex &gt;= 0 &amp;&amp; cityIndex &lt; 30 &amp;&amp; (State.citiesMask &amp; (1 &lt;&lt; cityInde
public static int UnlockedCityCount()
{
    int n = 0, mask = State.citiesMask;
    while (mask != 0) { n += mask &amp; 1; mask &gt;&gt;= 1; }
    return n;
}
/// &lt;summary&gt;Gold law: a captured gate rains one coin per attacker unit.&lt;/summary&gt;
public static int CoinsForGate(int units) =&gt; units &gt; 0 ? units * CoinPerUnit : 0;
// ---- daily gauntlet accounting (T016, G05 daily-cast law) ----
/// &lt;summary&gt;The daily gauntlet is settled when the marker equals the
/// shared day key - the next local midnight frees it by string
/// compare alone (zero reset beat).&lt;/summary&gt;
public static bool DailyUsedToday() =&gt; State.dailyKey == AdCapPolicy.TodayKey();
/// &lt;summary&gt;Captures booked for today's settled gauntlet (-1 = none).&lt;/summary&gt;
public static int DailyCapturesToday() =&gt; DailyUsedToday() ? State.dailyCaptures : -1;
/// &lt;summary&gt;Books the day's settled gauntlet: marker plus captures,
/// persisted on the booking beat; the first result of the day
/// stands - a same-day re-settle books nothing.&lt;/summary&gt;
public static void OnDailyChallenge(int captures)
{
    string dayKey = AdCapPolicy.TodayKey();
    if (string.IsNullOrEmpty(dayKey) || State.dailyKey == dayKey) return;
    State.dailyKey = dayKey;
    State.dailyCaptures = captures;
    Persist();
}
// ---- MiniJson round-trip ----
public static string Serialize(G07SaveModel m)
{
    var d = new Dictionary&lt;string, object&gt;
    {
        { "v", (long)m.v },
        { "coins", (long)m.coins },
```

```
                { "population", m.population },
                { "waveIndex", (long)m.waveIndex },
                { "citiesMask", (long)m.citiesMask },
                { "dailyKey", m.dailyKey },
                { "dailyCaptures", (long)m.dailyCaptures },
            };
            return MiniJson.Serialize(d);
        }
        public static G07SaveModel Deserialize(string json)
        {
            try
            {
                if (!(MiniJson.Deserialize(json) is Dictionary&lt;string, object&gt; d)) return null;
                var m = new G07SaveModel();
                m.v = GetInt(d, "v", 1);
                m.coins = GetInt(d, "coins", 0);
                m.population = GetLong(d, "population", 0);
                m.waveIndex = GetInt(d, "waveIndex", 0);
                m.citiesMask = GetInt(d, "citiesMask", 0);
                m.dailyKey = GetStr(d, "dailyKey", "");
                m.dailyCaptures = GetInt(d, "dailyCaptures", -1);
                return m;
            }
            catch { return null; }
        }
        static int GetInt(Dictionary&lt;string, object&gt; d, string key, int fallback)
            =&gt; d.TryGetValue(key, out var o) &amp;&amp; o is long l ? (int)l : fallback;
        static long GetLong(Dictionary&lt;string, object&gt; d, string key, long fallback)
            =&gt; d.TryGetValue(key, out var o) &amp;&amp; o is long l ? l : fallback;
        static string GetStr(Dictionary&lt;string, object&gt; d, string key, string fallback)
            =&gt; d.TryGetValue(key, out var o) &amp;&amp; o is string s ? s : fallback;
        static void CopyInto(G07SaveModel src, G07SaveModel dst)
        {
            dst.v = src.v;
            dst.coins = src.coins;
            dst.population = src.population;
            dst.waveIndex = src.waveIndex;
            dst.citiesMask = src.citiesMask;
            dst.dailyKey = src.dailyKey;
            dst.dailyCaptures = src.dailyCaptures;
        }
        static ISaveService ResolveRegistered()
        {
            return ServiceLocator.TryGet&lt;ISaveService&gt;(out var save) ? save : null;
        }
    }
}
// SplitValve.cs
// SplitValve.cs - three-way fork split valve (G07-T006, E4 graybox pincer model migrated).
// The player strokes a line across the fork and the tide peels into 1-3 columns: stroke
```

```
// dwell weights become lane units through a largest-remainder split (zero RNG, sum
// preserved), a multi-lane commit logs the route draw and opens the slow-mo window so
// the peel reads on screen (recordable share beat), and resolution is the frozen E4
// arithmetic: a single lane meets the converged guard line (guardPower * lanes *
// ConcentrationFactor - certain loss at table sizes), a split needs two lanes winning
// at once (encirclement). Counters keep the E4 names so the check re-runs the graybox
// ladder numbers exactly (33 single losses / 17 split captures over 200 waves).
// Pure C# with a settable logical clock - headless gates drive it deterministically.
using System;
using System.Collections.Generic;
namespace Sy.MobTide
{
    public enum AssaultOutcome { Captured, Lost }
    /// &lt;summary&gt;Route allocation for one assault - pure plan skill, no dice.&lt;/summary&gt;
    public sealed class RoutePlan
    {
        public int[] laneUnits;   // units committed per lane (length 1..3)
        public bool useLadder;    // wall waves (T007): stack a ladder instead of brute push
        public int TotalUnits()
        {
            int s = 0;
            if (laneUnits != null) for (int i = 0; i &lt; laneUnits.Length; i++) s += laneUnits[i];
            return s;
        }
    }
    public sealed class ValveEvent
    {
        public double t;
        public string type;
        public string detail;
    }
    public sealed class SplitValve
    {
        // E4 frozen pincer timeline (only-up law: these may only ever tighten).
        public const double ForkSeenAt = 0.30;        // fork becomes readable
        public const double RouteFlowAt = 0.50;       // columns stream down the lanes
        public const double PincerResolveDur = 3.00;  // fork -&gt; encirclement resolve
        // Combat law lives once in TideResolver (SYS04 authority, G07-T008);
        // these aliases keep the E4 vocabulary readable at the face.
        public const double ConcentrationFactor = TideResolver.ConcentrationFactor;
        public const int EncircleLanesRequired = TideResolver.EncircleLanesRequired;
        // T006 production beat: the slow-mo window on the split commit (readable =
        // recordable). Only-up: duration may lengthen, scale may only drop.
        public const double SlowMoDuration = 0.40;
        public const double SlowMoScale = 0.35;
        public double SimTime;
        public readonly List&lt;ValveEvent&gt; Events = new List&lt;ValveEvent&gt;();
        // In-run E4 counters (Stage2 names kept for the byte-parity re-run).
        public int PincerSingleAttempts, PincerSingleLosses;
        public int PincerSplitAttempts, PincerSplitCaptures;
```

```
public bool LastResolvedSlowMo;
public void Log(string type, string detail)
{
    Events.Add(new ValveEvent { t = SimTime, type = type, detail = detail });
}
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
/// &lt;summary&gt;Stroke-to-plan routing solver: per-lane dwell weights (any length,
/// zero/negative weight = lane not crossed) become integer lane units through the
/// largest-remainder method - deterministic, sum preserved exactly, ties broken
/// by lane order. An empty stroke keeps the whole tide in one column.&lt;/summary&gt;
public static RoutePlan PlanFromStroke(int units, double[] laneDwell)
{
    if (units &lt; 0) units = 0;
    var plan = new RoutePlan();
    if (laneDwell == null || laneDwell.Length == 0)
    {
        plan.laneUnits = new[] { units };
        return plan;
    }
    int lanes = Math.Min(laneDwell.Length, 3);
    double total = 0.0;
    for (int i = 0; i &lt; lanes; i++) if (laneDwell[i] &gt; 0.0) total += laneDwell[i];
    if (total &lt;= 0.0)
    {
        plan.laneUnits = new[] { units };
        return plan;
    }
    // Pass 1: exact floor shares for crossed lanes only (uncrossed lanes get 0).
    int crossed = 0;
    for (int i = 0; i &lt; lanes; i++) if (laneDwell[i] &gt; 0.0) crossed++;
    if (crossed &lt;= 1)
    {
        plan.laneUnits = new[] { units };
        return plan;
    }
    int[] laneUnits = new int[crossed];
    double[] rem = new double[crossed];
    int assigned = 0;
    int w = 0;
    for (int i = 0; i &lt; lanes; i++)
```

```
        {
            if (laneDwell[i] &lt;= 0.0) continue;
            double exact = units * laneDwell[i] / total;
            int floor = (int)Math.Floor(exact);
            laneUnits[w] = floor;
            rem[w] = exact - floor;
            assigned += floor;
            w++;
        }
        // Pass 2: largest-remainder hand-out, ties by earlier lane (deterministic).
        int left = units - assigned;
        while (left &gt; 0)
        {
            int best = 0;
            for (int i = 1; i &lt; crossed; i++) if (rem[i] &gt; rem[best]) best = i;
            laneUnits[best]++;
            rem[best] = -1.0; // spent - next largest wins
            left--;
        }
        plan.laneUnits = laneUnits;
        return plan;
    }
    /// &lt;summary&gt;One pincer fork assault (E4 frozen model). Single column: the guard
    /// line converges (concentration factor) and the push is repelled at table
    /// sizes. Split columns: each lane resolves alone, two lanes winning at once
    /// is the capture. Programming errors (null plan, wrong lane count, non-pincer
    /// spec) fail loud - this core only ever answers the fork face.&lt;/summary&gt;
    public AssaultOutcome Assault(WaveSpec spec, RoutePlan plan)
    {
        if (spec == null) throw new ArgumentNullException("spec");
        if (plan == null || plan.laneUnits == null || plan.laneUnits.Length &lt; 1 || plan.laneUnits.Length &gt; 3)
            throw new ArgumentException("route plan needs 1..3 lanes");
        if (spec.type != WaveType.Pincer) throw new ArgumentException("SplitValve resolves pincer forks only");
        double start = SimTime;
        Log("fork_seen", "three-way fork ahead");
        SimTime = start + ForkSeenAt;
        bool split = plan.laneUnits.Length &gt;= 2;
        LastResolvedSlowMo = false;
        if (split)
        {
            Log("split_draw", "route plan drawn: columns spread");
            Log("split_slowmo", "slow-mo window: duration=" + SlowMoDuration.ToString("0.00") +
                                "s scale=" + SlowMoScale.ToString("0.00"));
            LastResolvedSlowMo = true;
        }
        else
        {
            Log("column_push", "whole tide funneled into one lane");
        }
        SimTime = start + RouteFlowAt;
```

```
public const float FrameXMax = 360f;
public const float FrameYTop = -5f;
public const float FrameYBottom = -790f;
// Pixel floors (G05 caliber): the smallest must-see face must render
// its contracted dimension at no fewer than these device pixels.
public const float PxPerUnitFloor720 = 27f;
public const float PxPerUnitFloor1080 = 42f;
/// &lt;summary&gt;One must-see face contracted from the TideStageView
/// constants: center (center-origin x, top-anchored y) plus half
/// extents. Text faces measure their box height (fontSize + 20 per
/// MakeText) and a conservative glyph half-width; blocks contract as
/// squares of their MakeBlock size param.&lt;/summary&gt;
public readonly struct Face
{
    public readonly string name;
    public readonly float cx, cy, halfW, halfH;
    public Face(string name, float cx, float cy, float halfW, float halfH)
    {
        this.name = name;
        this.cx = cx;
        this.cy = cy;
        this.halfW = halfW;
        this.halfH = halfH;
    }
}
/// &lt;summary&gt;The must-see face list (frozen): every face the first
/// minute cannot lose to a notch or an aspect crop.&lt;/summary&gt;
public static List&lt;Face&gt; MustSeeFaces()
{
    return new List&lt;Face&gt;
    {
        // text faces: box height = fontSize + 20 (MakeText), glyphs stay centered
        new Face("CityBanner",       0f, -28f,  500f, 23f), // font 26
        new Face("PopulationText",   0f, -700f, 500f, 42f), // font 64
        new Face("PopulationLabel",  0f, -752f, 500f, 24f), // font 28
        // block faces: MakeBlock size params as square contracts
        new Face("SpawnGate",        0f, -420f,  80f,  80f), // size 160
        new Face("WallBlock",        0f, -560f, 160f, 160f), // size 320
        new Face("ForkGateL",     -300f, -560f,  60f,  60f), // size 120
        new Face("ForkGateC",        0f, -560f,  60f,  60f),
        new Face("ForkGateR",      300f, -560f,  60f,  60f),
    };
}
/// &lt;summary&gt;Drift probe face: the frozen frame values re-derived
/// from the frozen face list. Any face edit without a frame re-freeze
/// (or vice versa) breaks an equality here and the check catches it
/// the same round the layout changed.&lt;/summary&gt;
public static bool FacesConsistentWithFrame
{
    get
```

```
    {
        var faces = MustSeeFaces();
        // face order is frozen with the contract:
        // 0 CityBanner / 2 PopulationLabel / 5..7 fork gates L,C,R
        bool xOk = Mathf.Approximately(FrameXMin, faces[5].cx - faces[5].halfW)
                &amp;&amp; Mathf.Approximately(FrameXMax, faces[7].cx + faces[7].halfW);
        bool yTopOk = Mathf.Approximately(FrameYTop, faces[0].cy + faces[0].halfH);
        bool yBottomOk = FrameYBottom &lt;= faces[2].cy + faces[2].halfH + 1f;
        return xOk &amp;&amp; yTopOk &amp;&amp; yBottomOk;
    }
}
/// &lt;summary&gt;The fit plan for one device card: scale maps canvas units
/// to device px; safe bounds are in canvas coordinates (x center-origin,
/// y top-anchored growing downward, same system the layout uses).&lt;/summary&gt;
public struct FitPlan
{
    public float scale;
    public float safeLeft, safeRight;
    public float safeTop, safeBottom;
}
/// &lt;summary&gt;ScaleWithScreenSize fit for a screen + safe rect (device
/// px, y-down). A zero-size safe rect normalizes to the full screen
/// (null-safe for headless and odd devices). The y bounds are in the
/// LAYOUT convention the faces use (top-anchored: 0 = design top edge,
/// negative grows downward) so faces and safe rect compare directly.&lt;/summary&gt;
public static FitPlan Fit(float screenW, float screenH, Rect safePx)
{
    if (safePx.width &lt;= 0f || safePx.height &lt;= 0f)
        safePx = new Rect(0f, 0f, screenW, screenH);
    FitPlan p;
    p.scale = screenW / DesignWidth; // match-0 width bind (CanvasScaler agreement, check C1)
    p.safeLeft = (safePx.x - screenW * 0.5f) / p.scale;
    p.safeRight = p.safeLeft + safePx.width / p.scale;
    // layout y: distance below the design top edge, negated - a screen
    // taller than the design yields positive safeTop (slack above the
    // top edge), the safe bottom edge lands deep negative.
    p.safeTop = -DesignHeight * 0.5f + (screenH * 0.5f - safePx.y) / p.scale;
    p.safeBottom = p.safeTop - safePx.height / p.scale;
    return p;
}
/// &lt;summary&gt;Runtime face: re-center the built canvas on the device
/// safe area so the design frame centers on the safe center (a top
/// notch never eats the banner side). Null-safe by design - headless
/// gates, foreign canvases and missing scalers are a silent skip.&lt;/summary&gt;
public static void ApplyToCanvas()
{
    var scaler = UnityEngine.Object.FindObjectOfType&lt;CanvasScaler&gt;();
    if (scaler == null) return;
    if (scaler.referenceResolution.x != DesignWidth || scaler.referenceResolution.y != DesignHeight) return;
    var canvas = scaler.GetComponent&lt;Canvas&gt;();
```

```
            if (canvas == null || canvas.renderMode != RenderMode.ScreenSpaceOverlay) return;
            Rect safe = Screen.safeArea;
            if (safe.width &lt;= 0f || safe.height &lt;= 0f) return;
            Vector2 safeCenter = new Vector2(safe.x + safe.width * 0.5f, safe.y + safe.height * 0.5f);
            Vector2 screenCenter = new Vector2(Screen.width * 0.5f, Screen.height * 0.5f);
            Vector2 shiftPx = safeCenter - screenCenter;
            var rt = canvas.transform as RectTransform;
            if (rt == null) return;
            // screen px grow downward, RectTransform y grows upward - flip the y leg
            rt.anchoredPosition = rt.anchoredPosition + new Vector2(shiftPx.x, -shiftPx.y);
        }
    }
}
// TidePlaceholder.cs
// TidePlaceholder.cs - procedural placeholder faces for the tide stage (G07-T004).
// Art-first iron rule (X027): when ArtRuntime holds the stem the real sprite wins,
// otherwise the face falls back to a procedural color block - views render either
// way and no headless gate ever depends on art presence. Faction palette carries
// the E3#2 red line: the player tide is cobalt blue, the enemy is warm gray and
// night-steel, and no enemy face is ever red.
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
namespace Sy.MobTide
{
    public static class TidePlaceholder
    {
        // ---- faction palette (E3#2: enemy never red) ----
        public static readonly Color CrowdBlue = new Color(0.10f, 0.36f, 0.85f);
        public static readonly Color GuardGray = new Color(0.62f, 0.56f, 0.50f);
        public static readonly Color WallSteel = new Color(0.16f, 0.19f, 0.26f);
        public static readonly Color LaneSand = new Color(0.85f, 0.78f, 0.60f);
        public static readonly Color GateGold = new Color(0.95f, 0.78f, 0.25f);
        // S4 backdrop seat color (T025 slice b): the flat night block behind
        // the whole stage when the bg_stage art is absent (X027 fallback).
        public static readonly Color BackdropNight = new Color(0.06f, 0.09f, 0.15f);
        // ---- stems (art ships under these keys in S2.5; today they miss -&gt; fallback) ----
        public const string CrowdStem = "g07_crowd";
        public const string GuardStem = "g07_guard";
        public const string WallStem = "g07_wall";
        public const string LaneStem = "g07_lane";
        public const string GateStem = "g07_gate";
        // ---- T021 art-wave stems (B577c1 18-piece harvest, catalog-wired B581):
        // performance seats (ladder stack / shoe-drop laugh / goldrain / gatebreak
        // flash), HUD plate + operation icons, share-card base, stage backdrop.
        // Runtime seats land in the S4 wiring batch (performance acts, HUD
        // operation icons, share-card base, stage backdrop); until then every
        // key rides the same art-first iron rule via Face/UiFace (real art
        // wins, procedural fallback keeps gates art-independent).
        public const string LadderStem = "g07_ladder";
```

```
public const string ShoeStem = "g07_shoe";
public const string GoldrainStem = "g07_goldrain";
public const string GatebreakStem = "g07_gatebreak";
public const string PopcardStem = "g07_popcard";
public const string IconGateStem = "g07_icon_gate";
public const string IconSplitStem = "g07_icon_split";
public const string IconLadderStem = "g07_icon_ladder";
public const string IconRescueStem = "g07_icon_rescue";
public const string IconShareStem = "g07_icon_share";
public const string IconSetStem = "g07_icon_set";
public const string ShareBgStem = "g07_sharebg";
public const string BgStageStem = "g07_bg_stage";
// ---- UI chrome stems (T021 3c2: the AA-036 18-piece set ships under
// g07_ui_* keys; a missing stem falls back to a procedural semantic
// block so every UI seat renders either way - zero gate depends on art) ----
public const string UiBtnRound = "g07_ui_btn_round";
public const string UiBtnRoundDepth = "g07_ui_btn_round_depth";
public const string UiBtnRoundGrey = "g07_ui_btn_round_grey";
public const string UiBtnSquare = "g07_ui_btn_square";
public const string UiBtnRect = "g07_ui_btn_rect";
public const string UiCheckmark = "g07_ui_checkmark";
public const string UiCheckSquare = "g07_ui_check_square";
public const string UiCheckRound = "g07_ui_check_round";
public const string UiCross = "g07_ui_cross";
public const string UiCrossGrey = "g07_ui_cross_grey";
public const string UiCircle = "g07_ui_circle";
public const string UiSquare = "g07_ui_square";
public const string UiStar = "g07_ui_star";
public const string UiArrowN = "g07_ui_arrow_n";
public const string UiArrowE = "g07_ui_arrow_e";
public const string UiSlideH = "g07_ui_slide_h";
public const string UiSlideV = "g07_ui_slide_v";
public const string UiSlideHandle = "g07_ui_slide_handle";
static readonly Dictionary&lt;string, Sprite&gt; _cache = new Dictionary&lt;string, Sprite&gt;();
/// &lt;summary&gt;True when real art ships under the stem (S2.5 face - the
/// placeholder gates branch on this so art arrival never breaks them).&lt;/summary&gt;
public static bool ArtPresent(string stem) =&gt; ArtRuntime.Get(stem) != null;
/// &lt;summary&gt;The shipped art sprite for a stem (null on the placeholder
/// path) - editor gates compare against this without touching Sy.Core.&lt;/summary&gt;
public static Sprite ArtSprite(string stem) =&gt; ArtRuntime.Get(stem);
public static Sprite Crowd() =&gt; Face(CrowdStem, CrowdBlue, 32);
public static Sprite Guard() =&gt; Face(GuardStem, GuardGray, 32);
public static Sprite Wall() =&gt; Face(WallStem, WallSteel, 64);
public static Sprite Lane() =&gt; Face(LaneStem, LaneSand, 64);
public static Sprite Gate() =&gt; Face(GateStem, GateGold, 48);
/// &lt;summary&gt;Art-first with null-safe procedural fallback (X027 iron rule).&lt;/summary&gt;
public static Sprite Face(string stem, Color color, int px)
{
    Sprite art = ArtRuntime.Get(stem);
    if (art != null) return art;
```

```
    string key = stem + "#" + px;
    Sprite cached;
    if (_cache.TryGetValue(key, out cached) &amp;amp;&amp;amp; cached != null) return cached;
    Sprite made = ProceduralSprite(color, px);
    _cache[key] = made;
    return made;
}
/// &lt;summary&gt;UI chrome face (T021 3c2): art-first over the g07_ui_*
/// family, same iron rule as Face() - the caller passes the seat's
/// semantic fallback color (see UiFallbackColor).&lt;/summary&gt;
public static Sprite UiFace(string stem, Color color, int px)
{
    return Face(stem, color, px);
}
/// &lt;summary&gt;Semantic fallback color for a UI chrome seat (T021 3c2):
/// the procedural block keeps each seat's meaning readable when art is
/// absent - star = comedy gold, confirm/positive = cobalt, the grey
/// family keeps its grey semantics, slides stay neutral track grey.&lt;/summary&gt;
public static Color UiFallbackColor(string stem)
{
    switch (stem)
    {
        case UiBtnRoundDepth:
        case UiCross:
        case UiSquare: return WallSteel;
        case UiBtnRoundGrey:
        case UiCrossGrey:
        case UiSlideH:
        case UiSlideV: return GuardGray;
        case UiStar: return GateGold;
        default: return CrowdBlue;
    }
}
/// &lt;summary&gt;Procedural color block: an uncompressed readable px*px square.&lt;/summary&gt;
public static Sprite ProceduralSprite(Color color, int px)
{
    var tex = new Texture2D(px, px, TextureFormat.RGBA32, false);
    var pixels = new Color[px * px];
    for (int i = 0; i &lt; pixels.Length; i++) pixels[i] = color;
    tex.SetPixels(pixels);
    tex.Apply(false, false);
    tex.name = "sy_procedural_" + px;
    var sprite = Sprite.Create(tex, new Rect(0, 0, px, px), new Vector2(0.5f, 0.5f), px);
    sprite.name = tex.name;
    return sprite;
}
/// &lt;summary&gt;True when the enemy palette keeps the never-red red line
/// (both enemy faces hold more blue/green weight than red).&lt;/summary&gt;
public static bool EnemyNeverRed()
{
```

```
            return GuardGray.r &lt; GuardGray.g + GuardGray.b
                &amp;&amp; WallSteel.r &lt; WallSteel.g + WallSteel.b;
        }
    }
}
// TideResolver.cs
// TideResolver.cs - the single combat-resolution authority (G07-T008, SYS04).
// Every capture or wipe in the project flows through Decide/Resolve: the plain
// gate is raw lane power vs the guard line, a pincer fork needs two lanes
// winning at once (encirclement) while a single column meets the converged
// guard line (concentration factor - certain loss at table sizes), and a wall
// only ever falls to the ladder from behind (guard 999 law - a frontal brute
// push never wins). The decision face (Decide) is pure arithmetic with zero
// RNG; the flow face (Resolve) fronts a whole assault on the commit tick: it
// logs the wave, answers a capture with the gold-rain + fanfare + population
// triple on the very same beat (red-line-11 three-channel sync), folds the
// captured wave into the population counter (the HUD face) and emits
// evt_gate_captured with waveIdx+route; a wipe answers with the loss pair and
// the rescue offer slot. The choreography faces (TideStage/SplitValve/
// LadderSiege) delegate their decisions here so the combat law lives once.
// G07-T009 adds the rescue booking face (BookRescue, E4 Rescue law verbatim):
// the city-loss fail-retry converts into the population counter-attack - the
// comeback capture lands the capture triple on the surge beat and folds the
// rescued wave into the counter. The gate face (RescueGate) owns the ad
// category chain; this authority only books the combat answer.
using System;
using System.Collections.Generic;
namespace Sy.MobTide
{
    /// &lt;summary&gt;What one resolution decided - pure data, no events.&lt;/summary&gt;
    public struct ResolveVerdict
    {
        public AssaultOutcome outcome;
        public int lanesWon;    // pincer split: lanes that beat their own guard line
        public double effGuard; // the guard strength this push actually met
        public bool overTheTop; // wall: the ladder pours over (frontal line never applied)
    }
    public sealed class TideResolver
    {
        // E4 frozen combat law (only-up law: these may only ever tighten).
        public const double ConcentrationFactor = 2.0;  // guards converge on a single lane
        public const int EncircleLanesRequired = 2;     // pincer capture = &gt;= 2 lanes winning
        public const double RescueSurgeDur = 0.80;       // E4: the counter-attack surge before the retake
        public double SimTime;
        public readonly List&lt;ValveEvent&gt; Events = new List&lt;ValveEvent&gt;();
        public PopulationCounter Population { get; }
        // In-run counters (E4 Stage2 names kept for the replay face).
        public int Resolves, Captures, Losses;
        public int Rescues, RescuedCaptures;
        // Last-resolve state: the rescue window is the span between a booked
```

```
// city loss and the next resolve (or the rescue itself).
WaveSpec _lastSpec;
AssaultOutcome _lastOutcome;
bool _lastRescued;
public TideResolver() { Population = new PopulationCounter(); }
public TideResolver(PopulationCounter sharedPopulation) { Population = sharedPopulation; }
/// &lt;summary&gt;True right after a booked city loss and before the rescue
/// lands or the next wave resolves - the only window a rescue CTA may
/// even enter the ad chain.&lt;/summary&gt;
public bool RescueArmed
{
    get { return _lastOutcome == AssaultOutcome.Lost &amp;&amp; !_lastRescued; }
}
public void Log(string type, string detail)
{
    Events.Add(new ValveEvent { t = SimTime, type = type, detail = detail });
}
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
/// &lt;summary&gt;The decision face: deterministic combat arithmetic for any
/// wave type and any 1..3 lane plan. Zero RNG - same inputs, same
/// verdict, byte for byte. Programming errors fail loud.&lt;/summary&gt;
public static ResolveVerdict Decide(WaveSpec spec, RoutePlan plan)
{
    if (spec == null) throw new ArgumentNullException("spec");
    if (plan == null || plan.laneUnits == null || plan.laneUnits.Length &lt; 1 || plan.laneUnits.Length &gt; 3)
        throw new ArgumentException("route plan needs 1..3 lanes");
    var v = new ResolveVerdict { outcome = AssaultOutcome.Lost, lanesWon = 0 };
    if (spec.type == WaveType.Wall)
    {
        // Wall law: only the ladder over the top ever takes a wall -
        // guard 999 means a frontal push never wins at any table size.
        v.overTheTop = plan.useLadder;
        v.effGuard = spec.guardPower;
        if (plan.useLadder) v.outcome = AssaultOutcome.Captured;
        return v;
    }
    if (spec.type == WaveType.Plain)
    {
        // Plain gate: the whole tide against one guard line.
        v.effGuard = spec.guardPower;
```

```
            if (plan.TotalUnits() &gt; spec.guardPower) v.outcome = AssaultOutcome.Captured;
            return v;
        }
        // Pincer fork: split columns each meet their own guard line - two
        // lanes winning at once is the encirclement; a single column meets
        // the converged guard line instead (concentration factor).
        if (plan.laneUnits.Length &gt;= 2)
        {
            v.effGuard = spec.guardPower;
            for (int i = 0; i &lt; plan.laneUnits.Length &amp;&amp; i &lt; spec.lanes; i++)
                if (plan.laneUnits[i] &gt; spec.guardPower) v.lanesWon++;
            if (v.lanesWon &gt;= EncircleLanesRequired) v.outcome = AssaultOutcome.Captured;
            return v;
        }
        v.effGuard = spec.guardPower * spec.lanes * ConcentrationFactor;
        if (plan.laneUnits[0] &gt; v.effGuard) v.outcome = AssaultOutcome.Captured;
        return v;
    }
    /// &lt;summary&gt;One full assault on the commit tick: the wave logs in, the
    /// authority decides, and the answer lands on this same beat - a capture
    /// with the gold-rain + fanfare + count-up triple (three-channel sync)
    /// plus evt_gate_captured (waveIdx + route), a wipe with the loss pair
    /// and the rescue offer slot. The captured wave folds into the
    /// population counter - the HUD reads it live.&lt;/summary&gt;
    public AssaultOutcome Resolve(WaveSpec spec, RoutePlan plan)
    {
        ResolveVerdict verdict = Decide(spec, plan); // guards throw before any event
        Resolves++;
        Log("wave_start", "wave " + spec.waveIndex + " (" + spec.type + ") - " +
            RouteString(plan) + " meets guard " + verdict.effGuard.ToString("0.##"));
        if (verdict.outcome == AssaultOutcome.Captured)
        {
            Captures++;
            Log("gate_captured", "the gate falls");
            Log("gold_rain", "coins rain down");
            Log("sfx_victory", "fanfare");
            Population.AddUnits(spec.units);
            Log("count_up", "population=" + Population.RawValue());
            Log("evt_gate_captured", "waveIdx=" + spec.waveIndex + " route=" + RouteString(plan));
            _lastSpec = spec; _lastOutcome = AssaultOutcome.Captured; _lastRescued = false;
            return AssaultOutcome.Captured;
        }
        Losses++;
        Log("city_loss", "the wave is wiped out");
        Log("rescue_offer", "Reverse Redemption? (rewarded-ad fail-retry slot)");
        _lastSpec = spec; _lastOutcome = AssaultOutcome.Lost; _lastRescued = false;
        return AssaultOutcome.Lost;
    }
    /// &lt;summary&gt;"Reverse Redemption" booking (G07-T009, E4 Rescue law
    /// ported verbatim): valid only right after a booked city loss. The
```

```
        /// population surges back and retakes the gate - the rescued capture
        /// lands the capture triple (gold rain + fanfare + count-up) on the
        /// surge beat, folds the rescued wave into the population counter and
        /// counts as a capture (E4: Captures 143 = 142 direct + the rescued
        /// retry). The comeback is a scripted counter-attack, never a second
        /// resolve - zero RNG, same inputs, same comeback, byte for byte.&lt;/summary&gt;
        public bool BookRescue()
        {
            if (_lastSpec == null || _lastOutcome != AssaultOutcome.Lost || _lastRescued) return false;
            Rescues++;
            RescuedCaptures++;
            Captures++;
            _lastRescued = true;
            Log("rescue_accept", "Reverse Redemption accepted");
            Log("ad_break", "rewarded video");
            Log("counter_surge", "the population storms back");
            SimTime += RescueSurgeDur;
            Log("rescued_capture", "the tide retakes the gate");
            Log("gold_rain", "coins rain down");
            Log("sfx_victory", "fanfare");
            Population.AddUnits(_lastSpec.units);
            Log("count_up", "population=" + Population.RawValue());
            return true;
        }
        /// &lt;summary&gt;Route vocabulary for telemetry: how the tide went in.&lt;/summary&gt;
        public static string RouteString(RoutePlan plan)
        {
            if (plan == null || plan.laneUnits == null || plan.laneUnits.Length == 0) return "none";
            if (plan.useLadder) return "ladder";
            if (plan.laneUnits.Length == 1) return "single";
            return "split:" + plan.laneUnits.Length;
        }
    }
}
// TideShare.cs
// TideShare.cs - capture-beat share arming + compose + funnel (G07-T010).
// The 99,999 population brag card arms on the capture beat itself (same-tick
// law, G05-T009 ShareLoop precedent): every gate taken - direct or rescued -
// is a brag moment, and the card's face is the population counter (capped
// display 99,999 so the brag never shrinks). The resolver event stream stays
// read-only: every share face lives in this loop's own funnel, so bare-play
// streams stay byte-identical whether the player shares or not. Composition
// runs only on the player's own share tap (ReadPixels is a sync stall - never
// on an engine beat). Zero server: the query string is the whole payload,
// every parse and derive stays on-device (red line 12).
using System;
using System.Collections.Generic;
using Sy.Platform;
using Sy.Social;
namespace Sy.MobTide
```

```
{
    /// &lt;summary&gt;The share loop: arms the milestone channel on capture beats,
    /// composes the armed card through the shared pipeline.&lt;/summary&gt;
    public sealed class TideShare
    {
        public const string ChannelMilestone = "milestone";
        readonly TideResolver model;
        public readonly List&lt;string&gt; ShareFunnel = new List&lt;string&gt;();
        /// &lt;summary&gt;Armed channel: "milestone" after the latest capture beat,
        /// null before any arm. The latest arm wins; a loss beat never arms
        /// and never clears - the last brag stays shareable.&lt;/summary&gt;
        public string ShareChannel { get; private set; }
        public TideShare(TideResolver model) { this.model = model; }
        string Stamp()
        {
            return model == null ? "0.00" : model.SimTime.ToString("0.00");
        }
        static bool IsCaptureBeat(string type)
            =&gt; type == "gate_captured" || type == "rescued_capture";
        /// &lt;summary&gt;Capture-beat arming: the host calls this on every resolve
        /// tick (same-frame law). The latest beat decides: a capture (direct
        /// or rescued) arms the milestone channel and books the CTA the same
        /// tick; a loss beat is a no-op. A null or capture-less model never
        /// arms - never a throw.&lt;/summary&gt;
        public void NoteCapture()
        {
            if (model == null) return;
            bool captureBeat = false;
            for (int i = model.Events.Count - 1; i &gt;= 0; i--)
            {
                string t = model.Events[i].type;
                if (IsCaptureBeat(t)) { captureBeat = true; break; }
                if (t == "city_loss") break;
            }
            if (!captureBeat) return;
            ShareChannel = ChannelMilestone;
            ShareFunnel.Add("evt_share_cta_" + ShareChannel + "@" + Stamp());
        }
        /// &lt;summary&gt;Composes the armed channel's card through the shared
        /// pipeline. No armed context is a machine-readable fail, never a
        /// crash (the G04 guard law).&lt;/summary&gt;
        public ComposeResult ComposeCard(ShareCardService composer)
        {
            if (model == null || ShareChannel == null || composer == null)
            {
                return new ComposeResult { Ok = false, FailReason = "no_share_context" };
            }
            ComposeResult res = composer.Compose(TideCards.MilestoneCardFor(model, AdCapPolicy.TodayKey()));
            if (res.Ok)
                ShareFunnel.Add("evt_share_card@" + Stamp()
```

```
                + " pop=" + model.Population.DisplayValue());
        return res;
    }
    /// &lt;summary&gt;Test hygiene: clears the armed context and funnel.&lt;/summary&gt;
    public void Reset()
    {
        ShareChannel = null;
        ShareFunnel.Clear();
    }
}
/// &lt;summary&gt;Card spec factory (G07-T010): the 99,999 population brag card.
/// Composition stays 100% programmatic (AutoCardComposer flat builder,
/// null template = procedural dark card, headless stays green).&lt;/summary&gt;
public static class TideCards
{
    public const string WowTitle = "MY TIDE TOOK THE CITY!";
    public const string WowFooter = "CAN YOUR TIDE BEAT MINE?";
    public const string FestivalTitleSuffix = " HOLIDAY TRAVELERS TOOK THIS GATE!";
    /// &lt;summary&gt;Flat brag card: population (capped display face), gates
    /// taken and assaults led, plus the challenge footer. The query string
    /// is the whole share payload - zero server. The classic face is the
    /// frozen card (checks compose this one year-round).&lt;/summary&gt;
    public static ShareCard MilestoneCard(TideResolver r) =&gt; Build(r, WowTitle);
    /// &lt;summary&gt;Day-aware player card (G07-T032): inside the festival
    /// window the title brags in the holiday voice (population + travel
    /// meme), every other face - lines, footer, query - stays
    /// byte-identical; outside the window this is the classic card.&lt;/summary&gt;
    public static ShareCard MilestoneCardFor(TideResolver r, string dayKey)
    {
        if (r == null) return null;
        return CityRotation.IsFestival(dayKey)
            ? Build(r, r.Population.DisplayValue() + FestivalTitleSuffix)
            : Build(r, WowTitle);
    }
    static ShareCard Build(TideResolver r, string title)
    {
        if (r == null) return null;
        return AutoCardComposer.Card(
            null,
            title,
            new[]
            {
                "POPULATION: " + r.Population.DisplayValue(),
                "GATES TAKEN: " + r.Captures,
                "ASSAULTS LED: " + r.Resolves,
            },
            WowFooter,
            "src=share&amp;ch=milestone&amp;pop=" + r.Population.Value
                + "&amp;captures=" + r.Captures
                + "&amp;pushes=" + r.Resolves);
```

```
        }
    }
}
// TideSignalFaces.cs
// TideSignalFaces.cs - the three hook-signal presentation faces (G07-T020).
// The 10-second hook law (red line 11 / 06-script S0) demands one measurable
// signal from the laugh/wow/flex family inside the opening window; these faces
// are that signal layer, built after the G05-T025 SignalGate paradigm: pure C#
// presentation classes, no MonoBehaviour, code-built UGUI text under whatever
// parent the host hands in (the play view mounts them on the stage canvas,
// headless gates mount them on a bare root). The engine event streams stay
// read-only everywhere - the faces only drain them. Face clocks come from the
// host (stage clock in play mode, resolver clock in gates) so a signal stamps
// on the very beat the host presents it. First laugh: the shoe-drop beat flips
// the punch-line tag inside the opening window, bounded by its own life
// constant. First wow: the city-loss rescue offer flips the redemption tag on
// the loss tick itself and the accept dismisses it - the comeback supersedes
// the offer. First flex: every capture beat (direct or rescued) flips the
// population boast banner + CTA and books the evt_signal_boast funnel line; a
// loss beat never flips it. Zero RNG, zero server, English-only strings.
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.MobTide
{
    /// &lt;summary&gt;Laugh-family face: the shoe-drop punch line (G07-T020).&lt;/summary&gt;
    public sealed class ShoeLaughFace
    {
        public const string TagLine = "A SHOE?! HE KEEPS RUNNING!";
        public const float Life = 1.2f;               // bounded tag window (seconds)
        public const double OpeningWindowSec = 10.0;   // red line 11: first laugh &lt;= 10s
        public double FirstLaughAt { get; private set; } = -1.0;
        public int LaughBeats { get; private set; }
        Text _tag;
        Image _icon;
        int _drained;
        public int Rebuild(Transform parent)
        {
            _tag = SignalText.MakeText(parent, "laugh_tag", TagLine, 40, TidePlaceholder.GateGold);
            _tag.gameObject.SetActive(false);
            // 3c3 companion seat (seat spectrum A): the star flies at the
            // tag's left edge. The icon rides the tag itself - visibility
            // mirrors the tag's active window same-tick for free (parent
            // inactive hides children), so only its own activeSelf flips on.
            _icon = SignalIcon.MakeIcon(_tag.transform, "laugh_icon",
                TidePlaceholder.UiStar, 40, -320f);
            SignalIcon.AnchorBesideTag(_icon, -320f);
            _icon.gameObject.SetActive(true);
            _drained = 0;
            return 1;
```

```
    }
    /// &lt;summary&gt;Drains fresh stage events; the first shoe-drop beat stamps
    /// the laugh at the host clock the host presents it on.&lt;/summary&gt;
    public void NoteStage(TideStage stage, double now)
    {
        if (stage == null || _tag == null) return;
        for (int i = _drained; i &lt; stage.Events.Count; i++)
        {
            if (stage.Events[i].type != "shoe_drop") continue;
            if (FirstLaughAt &lt; 0.0) FirstLaughAt = now;
            LaughBeats++;
        }
        _drained = stage.Events.Count;
    }
    public void ApplyAt(double now)
    {
        if (_tag == null) return;
        bool live = FirstLaughAt &gt;= 0.0 &amp;&amp; now - FirstLaughAt &lt;= Life;
        _tag.gameObject.SetActive(live);
    }
}
/// &lt;summary&gt;Wow-family face: the city-loss reverse-redemption tag (G07-T020).
/// The offer books on the loss tick itself (TideResolver law); the tag
/// mirrors that beat and the accept dismisses it - the surge answer
/// supersedes the offer face.&lt;/summary&gt;
public sealed class RescueWowFace
{
    public const string TagLine = "THE GATE FELL - REVERSE REDEMPTION?";
    public const float HoldSec = 2.5f;             // bounded offer-tag window
    public double FirstWowAt { get; private set; } = -1.0;
    public double LastOfferAt { get; private set; } = -1.0;
    public int WowBeats { get; private set; }
    public bool Accepted { get; private set; }
    Text _tag;
    Image _icon;
    int _drained;
    public int Rebuild(Transform parent)
    {
        _tag = SignalText.MakeText(parent, "wow_tag", TagLine, 44, TidePlaceholder.CrowdBlue);
        _tag.gameObject.SetActive(false);
        // 3c4 eyeball adjudication: the plain checkmark carries the
        // confirm read at 32px (the check_round's stroke vanished
        // blue-on-blue at wired size - B538 preview panel evidence).
        _icon = SignalIcon.MakeIcon(_tag.transform, "wow_icon",
            TidePlaceholder.UiCheckmark, 32, -320f);
        SignalIcon.AnchorBesideTag(_icon, -320f);
        _icon.gameObject.SetActive(true);
        _drained = 0;
        return 1;
    }
```

```
    /// &lt;summary&gt;Drains fresh resolver events; every rescue-offer beat stamps
    /// the wow at the host clock, the rescue accept closes the tag.&lt;/summary&gt;
    public void NoteResolver(TideResolver resolver, double now)
    {
        if (resolver == null || _tag == null) return;
        for (int i = _drained; i &lt; resolver.Events.Count; i++)
        {
            string t = resolver.Events[i].type;
            if (t == "rescue_offer")
            {
                if (FirstWowAt &lt; 0.0) FirstWowAt = now;
                LastOfferAt = now;
                WowBeats++;
            }
            else if (t == "rescue_accept") Accepted = true;
        }
        _drained = resolver.Events.Count;
    }
    public void ApplyAt(double now)
    {
        if (_tag == null) return;
        bool live = !Accepted &amp;&amp; LastOfferAt &gt;= 0.0 &amp;&amp; now - LastOfferAt &lt;= HoldSec;
        _tag.gameObject.SetActive(live);
    }
}
/// &lt;summary&gt;Flex-family face: the population boast card (G07-T020).
/// Every capture beat (direct or rescued) flips the banner + CTA and books
/// the evt_signal_boast funnel line; a loss beat never flips it.&lt;/summary&gt;
public sealed class BoastFlexFace
{
    public const string BannerSuffix = " PEOPLE TOOK THIS GATE!";
    public const string CtaLine = "BRAG ABOUT IT - SHARE THE CARD";
    public const float HoldSec = 1.5f;             // bounded banner window
    public int FlexBeats { get; private set; }
    public double LastCaptureAt { get; private set; } = -1.0;
    public readonly List&lt;string&gt; Funnel = new List&lt;string&gt;();
    Text _banner;
    Text _cta;
    Image _icon;
    int _drained;
    public int Rebuild(Transform parent)
    {
        _banner = SignalText.MakeText(parent, "boast_banner", "", 52, TidePlaceholder.GateGold);
        _cta = SignalText.MakeText(parent, "boast_cta", CtaLine, 30, TidePlaceholder.WallSteel);
        _banner.gameObject.SetActive(false);
        _cta.gameObject.SetActive(false);
        // 3c3 companion seat: the arrow_n flies at the banner's left
        // edge, riding the banner's active window (see ShoeLaughFace).
        _icon = SignalIcon.MakeIcon(_banner.transform, "boast_icon",
            TidePlaceholder.UiArrowN, 32, -360f);
```

```
            SignalIcon.AnchorBesideTag(_icon, -360f);
            _icon.gameObject.SetActive(true);
            _drained = 0;
            return 2;
        }
        /// &lt;summary&gt;Drains fresh resolver events; every capture beat (direct or
        /// rescued) is a brag moment - the banner face reads the folded
        /// population counter on that very beat.&lt;/summary&gt;
        public void NoteResolver(TideResolver resolver, double now)
        {
            if (resolver == null || _banner == null) return;
            for (int i = _drained; i &lt; resolver.Events.Count; i++)
            {
                string t = resolver.Events[i].type;
                if (t != "gate_captured" &amp;&amp; t != "rescued_capture") continue;
                FlexBeats++;
                LastCaptureAt = now;
                _banner.text = resolver.Population.DisplayValue() + BannerSuffix;
                Funnel.Add("evt_signal_boast@" + now.ToString("0.00"));
            }
            _drained = resolver.Events.Count;
        }
        public void ApplyAt(double now)
        {
            if (_banner == null) return;
            bool live = FlexBeats &gt; 0 &amp;&amp; now - LastCaptureAt &lt;= HoldSec;
            _banner.gameObject.SetActive(live);
            if (_cta != null) _cta.gameObject.SetActive(live);
        }
    }
    /// &lt;summary&gt;Companion icon builder for the signal faces (T021 3c2 seat
    /// skeleton): an additive UGUI Image flying next to a signal tag. The tag
    /// text bytes stay untouched (SignalGateCheck zero-drift contract), the
    /// icon never eats taps, and the host face mirrors its tag's active
    /// window onto the icon when 3c3 wires the seats.&lt;/summary&gt;
    static class SignalIcon
    {
        /// &lt;summary&gt;Builds a companion icon on the tag row: art-first UI
        /// chrome with a procedural semantic fallback, native size then one
        /// uniform scale so the widest dimension lands on px. Starts hidden -
        /// companion icons wake with their host tag.&lt;/summary&gt;
        public static Image MakeIcon(Transform parent, string name, string stem, int px, float offsetX)
        {
            var go = new GameObject(name, typeof(Image));
            go.transform.SetParent(parent, false);
            var image = go.GetComponent&lt;Image&gt;();
            image.sprite = TidePlaceholder.UiFace(stem, TidePlaceholder.UiFallbackColor(stem), px);
            image.raycastTarget = false;   // the signal layer never eats taps
            var rt = image.rectTransform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 1f);
```

```
            rt.anchoredPosition = new Vector2(offsetX, -200f);
            image.SetNativeSize();
            float w = image.sprite != null ? image.sprite.rect.width : 0f;
            if (w &gt; 0f) rt.localScale = Vector3.one * (px / w);
            go.SetActive(false);
            return image;
        }
        /// &lt;summary&gt;3c3 seat anchor: re-anchors a tag-child icon onto the
        /// host tag's midline - MakeIcon's y=-200 targets the face root, but
        /// a tag-riding icon must sit on the tag row whatever the host
        /// mounts (zero-drift: the tag's own rect is the only frame).&lt;/summary&gt;
        public static void AnchorBesideTag(Image icon, float offsetX)
        {
            var rt = icon.rectTransform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = new Vector2(offsetX, 0f);
        }
    }
    /// &lt;summary&gt;Shared code-built UGUI text factory for the signal faces -
    /// same conventions as the stage view (builtin font, anchored midtop).&lt;/summary&gt;
    static class SignalText
    {
        public static Text MakeText(Transform parent, string name, string content, int size, Color color)
        {
            var go = new GameObject(name, typeof(Text));
            go.transform.SetParent(parent, false);
            var text = go.GetComponent&lt;Text&gt;();
            text.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            text.fontSize = size;
            text.color = color;
            text.alignment = TextAnchor.MiddleCenter;
            text.horizontalOverflow = HorizontalWrapMode.Overflow;
            text.verticalOverflow = VerticalWrapMode.Overflow;
            var rt = text.rectTransform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 1f);
            rt.anchoredPosition = new Vector2(0f, -200f);
            rt.sizeDelta = new Vector2(1000f, size + 20f);
            text.text = content;
            text.raycastTarget = false;   // the signal layer never eats taps
            return text;
        }
    }
}
// TideStage.cs
// TideStage.cs - first-minute assault stage core (G07-T005, E4 Stage1 timeline migrated).
// The opening beats the graybox froze, now driving the real project: hook caption at
// 0.15s, first tap at 0.30s answers with the stream+sfx+count triple on the very same
// tick (red line 11: first operation must answer within 1s - here delta is 0), the
// plain charge starts 0.20s after the tap, the first charger drops a shoe at +0.70s
// (first laugh), and the first gate falls at +2.00s folding the wave into the
```

```
// population counter. Zero RNG: the opening wave is WaveTable.SpecFor(0).
// Pure C# with an injected clock - headless gates drive it deterministically, the
// view feeds it Time.deltaTime in play mode.
using System;
using System.Collections.Generic;
namespace Sy.MobTide
{
    public enum StagePhase { Caption, AwaitingTap, Charging, Captured }
    public sealed class StageEvent
    {
        public double t;
        public string type;
        public string detail;
    }
    public sealed class TideStage
    {
        // E4 Stage1 baselines (only-up law: these may only ever tighten).
        public const double CaptionAt = 0.15;          // hook caption slot (measured 0.15, promise &lt;= 3s)
        public const double ChargeDelayAfterTap = 0.20; // stream pours, then the charge commits
        public const double PlainChargeDur = 2.00;     // gate falls this long after charge start
        public const double FirstShoeOffset = 0.70;    // first laugh inside the charge
        public readonly List&lt;StageEvent&gt; Events = new List&lt;StageEvent&gt;();
        public StagePhase Phase { get; private set; }
        public double SimTime { get; private set; }
        public PopulationCounter Population { get; } = new PopulationCounter();
        public WaveSpec Wave { get; } = WaveTable.SpecFor(WaveTable.FirstWave);
        public bool ShoeDropped { get; private set; }
        double _chargeStart = -1.0;
        bool _shoeQueued;
        public TideStage() { Phase = StagePhase.Caption; _shoeQueued = true; }
        /// &lt;summary&gt;Capture booking seam (T012): null = unwired (headless S1
        /// gates stay byte-identical); the play-mode host wires it to
        /// SaveWire.OnGateCaptured so the meta face persists on the capture
        /// beat itself, never on a timer.&lt;/summary&gt;
        public static Action&lt;WaveSpec&gt; CaptureBooker;
        public void Log(string type, string detail)
        {
            Events.Add(new StageEvent { t = SimTime, type = type, detail = detail });
        }
        public double TimeOf(string type)
        {
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
            return -1.0;
        }
        public int CountEvents(string type)
        {
            int n = 0;
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
            return n;
        }
```

```
/// &lt;summary&gt;Clock tick: fires the hook caption in its slot, then drives the
/// charge timeline (shoe, first capture) once the tap opened the gate.
/// Event order inside one tick is fixed: caption -&gt; shoe -&gt; capture.&lt;/summary&gt;
public void Advance(double dt)
{
    if (dt &lt; 0) dt = 0;
    double target = SimTime + dt;
    if (Phase == StagePhase.Caption &amp;&amp; target &gt;= CaptionAt)
    {
        SimTime = CaptionAt;
        Log("caption", "The tide obeys one finger.");
        Phase = StagePhase.AwaitingTap;
    }
    if (Phase == StagePhase.Charging)
    {
        if (_shoeQueued &amp;&amp; target &gt;= _chargeStart + FirstShoeOffset)
        {
            SimTime = _chargeStart + FirstShoeOffset;
            Log("shoe_drop", "a charger's shoe flies off - keeps running barefoot");
            ShoeDropped = true;
            _shoeQueued = false;
        }
        if (target &gt;= _chargeStart + PlainChargeDur)
        {
            SimTime = _chargeStart + PlainChargeDur;
            ResolveWave();
        }
    }
    SimTime = target;
}
/// &lt;summary&gt;First core operation: tap the spawn gate. The stream, the whoosh
/// and the population pop answer on the very same tick (three-channel sync).&lt;/summary&gt;
public bool TapGate()
{
    if (Phase != StagePhase.AwaitingTap) return false;
    Log("first_tap", "spawn gate tapped");
    Log("stream_flow", "the tide pours out");
    Log("sfx_gush", "whoosh");
    Log("count_pop", "population counter starts climbing");
    Phase = StagePhase.Charging;
    _chargeStart = SimTime + ChargeDelayAfterTap;
    return true;
}
void ResolveWave()
{
    Phase = StagePhase.Captured;
    // Decision delegated to the SYS04 authority (G07-T008) - the plain
    // gate law (whole tide vs one guard line) lives in TideResolver.
    if (TideResolver.Decide(Wave, SplitValve.SingleRoutePlan(Wave.units)).outcome == AssaultOutcome.Captured)
    {
```

```
                Log("gate_captured", "the gate falls");
                Log("gold_rain", "coins rain down");
                Log("sfx_victory", "fanfare");
                Population.AddUnits(Wave.units);
                Log("count_up", "population=" + Population.RawValue());
                var booker = CaptureBooker;
                if (booker != null) booker(Wave);   // T012: capture beat books the meta save
            }
            else
            {
                Log("city_loss", "the wave is wiped out");
                Log("rescue_offer", "Reverse Redemption? (rewarded-ad fail-retry slot)");
            }
        }
    }
}
// TideStageView.cs
// TideStageView.cs - placeholder-view face of the first-minute stage (G07-T005).
// Builds its whole canvas in code (no scene authoring yet): hook caption, the
// tappable gate block, a stream of crowd blocks, and the live population card.
// Faces follow the X027 art-first law - when real art ships in S2.5 under the
// TidePlaceholder stems it wins automatically, the layout does not care.
// The audio channel is event-driven today (sfx_gush logs through the stage);
// the AudioManager subscribes the same events when the audio batch lands.
using System.Collections.Generic;
using Sy.Platform;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;
namespace Sy.MobTide
{
    public sealed class TideStageView : MonoBehaviour
    {
        const int StreamCount = 8;
        const float StreamSpeed = 240f;
        TideStage _stage;
        TideResolver _resolver;
        Transform _canvas;
        Text _caption;
        Text _population;
        Image _gate;
        Image _shoe;
        Vector2 _shoeVel;
        readonly List&lt;Image&gt; _stream = new List&lt;Image&gt;();
        readonly List&lt;StageEvent&gt; _pending = new List&lt;StageEvent&gt;();
        int _drained;
        // S4 wiring slice b (T025): the capture-act seats (gatebreak flash +
        // gold rain), the boast-card base and the stage backdrop - all
        // art-first (X027), all additive view faces riding the frozen beats;
        // the engine streams and the face classes stay untouched.
```

```
Image _boastPlate;
Image _gatebreak;
float _gatebreakLife;
readonly List&lt;Image&gt; _goldrain = new List&lt;Image&gt;();
readonly List&lt;Vector2&gt; _goldVel = new List&lt;Vector2&gt;();
const float GatebreakLife = 0.60f;
const int GoldrainPerBurst = 12;
const int GoldrainCap = 36;
// Fork face (T006): after the wall falls the three-way fork fades in,
// the player strokes across the lanes (dwell weights), the commit peels the
// stream into columns and the slow-mo window makes the peel readable.
const int FirstPincerWave = 3;   // teaching rotation: first fork is wave index 3
const float ForkGateX = 300f;
readonly Image[] _forkGates = new Image[3];
readonly double[] _dwell = new double[3];
Text _forkHint;
Text _forkResult;
bool _forkBuilt, _stroking, _forkResolved;
float _slowMoLeft;
// Wall face (T007): the first wall drops right after the opening gate -
// forced teaching, hold anywhere to stack the human ladder: three lock-arm
// layers light up inside the 1.50s window, the tide pours over the top and
// takes the gate from behind. Brute pushing is always available and always
// loses (guard 999) - the wall teaches the ladder verb.
const int FirstWallWave = 1;     // teaching rotation: first wall is wave index 1 (E4 wave2)
LadderSiege _siege;
Image _wallBlock;
Text _wallHint;
Text _wallResult;
readonly Image[] _lockLayers = new Image[LadderSiege.LockArmLayers];
bool _wallBuilt, _wallResolved;
double _holdHeld;
// Hook-signal faces (T020): the laugh/wow/flex layer - pure C# faces
// mounted under the same canvas; they drain the engine streams
// themselves, the view only hands them the stage clock every frame.
ShoeLaughFace _laughFace;
RescueWowFace _wowFace;
BoastFlexFace _flexFace;
void Start()
{
    _stage = new TideStage();
    // SYS04 fronts the play flow (G07-T008): wall and fork commits
    // resolve through the single authority, which folds captured waves
    // into the shared population counter - one HUD source of truth.
    _resolver = new TideResolver(_stage.Population);
    // T012 local-authority save: boot reads the slot, the capture beat
    // books the meta face through the seam the stage exposes.
    SaveWire.LoadOrDefault();
    TideStage.CaptureBooker = SaveWire.OnGateCaptured;
    BuildUi();
```

```
}
void Update()
{
    _stage.Advance(Time.deltaTime);
    DrainEvents();
    // T020: the signal faces ride the stage clock (the presentation
    // clock) - a signal stamps on the very frame its beat drains.
    if (_laughFace != null) { _laughFace.NoteStage(_stage, _stage.SimTime); _laughFace.ApplyAt(_stage.SimTime);
    if (_wowFace != null) { _wowFace.NoteResolver(_resolver, _stage.SimTime); _wowFace.ApplyAt(_stage.SimTime);
    if (_flexFace != null) { _flexFace.NoteResolver(_resolver, _stage.SimTime); _flexFace.ApplyAt(_stage.SimTime
    // S4 wiring slice b: the plate mirrors the boast window exactly
    // as the frozen face derives it (same public fields, no face
    // touch), and the capture acts tick every frame.
    if (_boastPlate != null)
        _boastPlate.gameObject.SetActive(_flexFace != null
            &amp;&amp; _flexFace.FlexBeats &gt; 0
            &amp;&amp; _stage.SimTime - _flexFace.LastCaptureAt &lt;= BoastFlexFace.HoldSec);
    TickCaptureActs();
    if (_stage.Phase == StagePhase.Charging || _stage.Phase == StagePhase.Captured) TickStream();
    if (_shoe != null) TickShoe();
    if (_stage.Phase == StagePhase.Captured &amp;&amp; !_wallBuilt) BuildWall();
    if (_wallBuilt &amp;&amp; !_wallResolved) TickWallHold();
    if (_wallBuilt &amp;&amp; _wallResolved &amp;&amp; !_forkBuilt) BuildFork();
    if (_forkBuilt &amp;&amp; !_forkResolved) TickStroke();
    if (_slowMoLeft &gt; 0f)
    {
        _slowMoLeft -= Time.unscaledDeltaTime;
        if (_slowMoLeft &lt;= 0f) Time.timeScale = 1f;
    }
}
void BuildUi()
{
    // Known-infrastructure red line: a UGUI scene must never end up
    // without an EventSystem, so the view self-heals it on boot.
    if (FindObjectOfType&lt;EventSystem&gt;() == null)
    {
        var es = new GameObject("EventSystem", typeof(EventSystem), typeof(StandaloneInputModule));
        DontDestroyOnLoad(es); // survives the boot -&gt; main scene switch
    }
    var canvasGo = new GameObject("TideCanvas", typeof(Canvas), typeof(CanvasScaler), typeof(GraphicRaycaster));
    var canvas = canvasGo.GetComponent&lt;Canvas&gt;();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    var scaler = canvasGo.GetComponent&lt;CanvasScaler&gt;();
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(1080, 1440);
    _canvas = canvasGo.transform;
    // S4 wiring slice b: the stage backdrop seat - the bg_stage art
    // fills the frozen design frame in both paths (the art is 3:4,
    // the exact 1080x1440 frame aspect, so the stretch never
    // distorts; the fallback reads as a flat night block), mounted
```

```
// first so it draws behind every face, and it never eats taps.
var backdrop = MakeBlock(canvas.transform, "StageBackdrop",
    TidePlaceholder.Face(TidePlaceholder.BgStageStem, TidePlaceholder.BackdropNight, 64),
    new Vector2(0, -720), 1080);
backdrop.rectTransform.localScale = Vector3.one;
backdrop.rectTransform.sizeDelta = new Vector2(1080, 1440);
backdrop.raycastTarget = false;
_caption = MakeText(canvas.transform, "CaptionText", "The tide obeys one finger.",
    new Vector2(0, -80), 56, TidePlaceholder.CrowdBlue);
_caption.gameObject.SetActive(false); // fires with the caption event
// LiveOps day banner (G07-T015): the day's front city and garrison
// theme - a face the rotation owns alone, every existing face stays
// untouched (the derive is pure, the engine never reads it).
// Festival window (G07-T032): inside 10-01..10-08 the line and its
// accent wear the HOLIDAY TIDE skin - same single banner source.
MakeText(canvas.transform, "CityBanner", CityRotation.BannerText(AdCapPolicy.TodayKey()),
    new Vector2(0, -28), 26, HexColor(CityRotation.BannerAccentHex(AdCapPolicy.TodayKey())));
// S4 wiring slice a (T021 seat harvest): the population HUD rides
// the popcard plate - art-first, and the procedural fallback is a
// flat dark card so the seat stays readable either way (X027).
// Fixed 320x160 box (2:1 popcard aspect) keeps the plate inside
// the frozen design frame in both art-present and fallback paths.
var popcard = MakeBlock(canvas.transform, "PopulationCard",
    TidePlaceholder.Face(TidePlaceholder.PopcardStem, TidePlaceholder.WallSteel, 160),
    new Vector2(0, -700), 320);
popcard.rectTransform.localScale = Vector3.one;
popcard.rectTransform.sizeDelta = new Vector2(320, 160);
_population = MakeText(canvas.transform, "PopulationText", "0",
    new Vector2(0, -700), 64, TidePlaceholder.GateGold);
MakeText(canvas.transform, "PopulationLabel", "POPULATION",
    new Vector2(0, -752), 28, TidePlaceholder.WallSteel);
_gate = MakeBlock(canvas.transform, "SpawnGate", TidePlaceholder.Gate(),
    new Vector2(0, -420), 160);
var gateButton = _gate.gameObject.AddComponent&lt;Button&gt;();
gateButton.targetGraphic = _gate;
gateButton.onClick.AddListener(OnGateTapped);
for (int i = 0; i &lt; StreamCount; i++)
{
    var block = MakeBlock(canvas.transform, "Crowd" + i, TidePlaceholder.Crowd(),
        new Vector2(0, -420), 40);
    block.gameObject.SetActive(false);
    _stream.Add(block);
}
// S4 wiring slice b: the six operation-icon menu seats - the
// icon_* stems land art-first (semantic color fallbacks keep
// each seat readable), a passive legend strip that never eats
// taps: the gestures stay the only verbs, zero new beats.
string[] opIcons = { TidePlaceholder.IconGateStem, TidePlaceholder.IconSplitStem,
    TidePlaceholder.IconLadderStem, TidePlaceholder.IconRescueStem,
    TidePlaceholder.IconShareStem, TidePlaceholder.IconSetStem };
```

```
    Color[] iconFallback = { TidePlaceholder.GateGold, TidePlaceholder.CrowdBlue,
        TidePlaceholder.CrowdBlue, TidePlaceholder.GateGold, TidePlaceholder.GateGold,
        TidePlaceholder.WallSteel };
    for (int i = 0; i &lt; opIcons.Length; i++)
    {
        var icon = MakeBlock(canvas.transform, "OpIcon" + i,
            TidePlaceholder.Face(opIcons[i], iconFallback[i], 84),
            new Vector2((i - 2.5f) * 120f, -1310f), 84);
        icon.raycastTarget = false;
    }
    // S4 wiring slice b: the boast-card base seat - the sharebg art
    // carries the population boast as a share-card preview (3:4
    // frozen box in both paths), parked one slot above the backdrop
    // so every HUD face still draws over it; the active window is
    // mirrored in Update from the frozen face's public fields, the
    // face class itself stays gate-locked (zero drift).
    _boastPlate = MakeBlock(canvas.transform, "BoastCardBase",
        TidePlaceholder.Face(TidePlaceholder.ShareBgStem, TidePlaceholder.BackdropNight, 160),
        new Vector2(0, -480), 540);
    _boastPlate.rectTransform.localScale = Vector3.one;
    _boastPlate.rectTransform.sizeDelta = new Vector2(540, 720);
    _boastPlate.raycastTarget = false;
    _boastPlate.transform.SetSiblingIndex(1);
    _boastPlate.gameObject.SetActive(false);
    // T020 signal layer: the laugh/wow/flex faces mount last - zero
    // touch on any existing beat, the engine streams stay read-only.
    _laughFace = new ShoeLaughFace();
    _laughFace.Rebuild(canvas.transform);
    _wowFace = new RescueWowFace();
    _wowFace.Rebuild(canvas.transform);
    _flexFace = new BoastFlexFace();
    _flexFace.Rebuild(canvas.transform);
    // T023: re-center the canvas on the device safe area - the fit
    // contract lives in TideDeviceFit (manual 6.1).
    TideDeviceFit.ApplyToCanvas();
}
void OnGateTapped()
{
    if (_stage.TapGate()) DrainEvents();
}
void DrainEvents()
{
    _pending.Clear();
    // fresh events only - the stage never rewrites history
    for (int i = _drained; i &lt; _stage.Events.Count; i++) _pending.Add(_stage.Events[i]);
    _drained = _stage.Events.Count;
    foreach (var e in _pending)
    {
        switch (e.type)
        {
```

```
            case "caption":
                if (_caption != null) _caption.gameObject.SetActive(true);
                break;
            case "count_pop":
            case "count_up":
                if (_population != null) _population.text = _stage.Population.DisplayValue();
                break;
            case "shoe_drop":
                SpawnShoe();
                break;
            case "gate_captured":
                if (_gate != null) _gate.color = TidePlaceholder.CrowdBlue; // tide owns the gate
                FireCaptureActs();
                break;
        }
    }
}
void TickStream()
{
    float laneHalf = 90f;
    for (int i = 0; i &lt; _stream.Count; i++)
    {
        var block = _stream[i];
        if (!block.gameObject.activeSelf) continue;
        var pos = block.rectTransform.anchoredPosition;
        pos.y += StreamSpeed * Time.deltaTime;
        if (pos.y &gt; 620f) pos.y = -420f; // loop back to the gate
        block.rectTransform.anchoredPosition = pos;
    }
    // gate open -&gt; blocks start pouring one frame apart (staggered feel)
    if (_stage.Phase == StagePhase.Charging)
        for (int i = 0; i &lt; _stream.Count; i++)
        {
            var block = _stream[i];
            if (!block.gameObject.activeSelf)
            {
                var pos = block.rectTransform.anchoredPosition;
                pos.x = ((i % 2) * 2 - 1) * (i / 2) * (laneHalf / StreamCount);
                pos.y = -420f;
                block.rectTransform.anchoredPosition = pos;
                block.gameObject.SetActive(true);
                break; // one per frame keeps the pour readable
            }
        }
}
void SpawnShoe()
{
    if (_shoe != null) return;
    _shoe = MakeBlock(_canvas, "Shoe",
        // S4 wiring slice a: the shoe seat takes the real shoe art
```

```
            // (art-first) instead of riding the gate sprite - the laugh
            // beat finally drops a shoe, not a gold gate block.
            TidePlaceholder.Face(TidePlaceholder.ShoeStem, TidePlaceholder.GateGold, 28),
            new Vector2(60, -360), 28);
        _shoeVel = new Vector2(320f, 520f);
    }
    void TickShoe()
    {
        _shoeVel.y -= 900f * Time.deltaTime; // gravity arc - the laugh beat
        var pos = _shoe.rectTransform.anchoredPosition + _shoeVel * Time.deltaTime;
        _shoe.rectTransform.anchoredPosition = pos;
        var c = _shoe.color;
        c.a = Mathf.Max(0f, c.a - 0.25f * Time.deltaTime);
        _shoe.color = c;
        if (c.a &lt;= 0f || pos.y &lt; -900f) Destroy(_shoe.gameObject);
    }
    // ---- S4 wiring slice b: capture acts (gatebreak flash + gold rain) ----
    void FireCaptureActs()
    {
        // The capture celebration finally looks like itself: the
        // gatebreak art flashes over the fallen gate and a gold rain
        // pours - deterministic index spread, zero RNG on the face
        // (X027 art-first both, fallback keeps the beats readable).
        if (_gatebreak == null)
        {
            _gatebreak = MakeBlock(_canvas, "GatebreakFlash",
                TidePlaceholder.Face(TidePlaceholder.GatebreakStem, TidePlaceholder.CrowdBlue, 160),
                new Vector2(0, -420), 160);
            _gatebreak.raycastTarget = false;
            _gatebreakLife = GatebreakLife;
        }
        for (int i = 0; i &lt; GoldrainPerBurst &amp;&amp; _goldrain.Count &lt; GoldrainCap; i++)
        {
            float x = ((i * 137) % 900) - 450;
            var drop = MakeBlock(_canvas, "Goldrain" + _goldrain.Count,
                TidePlaceholder.Face(TidePlaceholder.GoldrainStem, TidePlaceholder.GateGold, 24),
                new Vector2(x, 60f - (i % 4) * 40f), 24);
            drop.raycastTarget = false;
            _goldrain.Add(drop);
            _goldVel.Add(new Vector2(0f, -(260f + (i % 5) * 60f)));
        }
    }
    void TickCaptureActs()
    {
        if (_gatebreak != null)
        {
            _gatebreakLife -= Time.deltaTime;
            var flash = _gatebreak.color;
            flash.a = Mathf.Clamp01(_gatebreakLife / GatebreakLife);
            _gatebreak.color = flash;
```

```
            if (_gatebreakLife &lt;= 0f) { Destroy(_gatebreak.gameObject); _gatebreak = null; }
        }
        for (int i = _goldrain.Count - 1; i &gt;= 0; i--)
        {
            var vel = _goldVel[i];
            vel.y -= 700f * Time.deltaTime;
            _goldVel[i] = vel;
            var drop = _goldrain[i];
            var pos = drop.rectTransform.anchoredPosition + vel * Time.deltaTime;
            drop.rectTransform.anchoredPosition = pos;
            var c = drop.color;
            c.a = Mathf.Max(0f, c.a - 0.30f * Time.deltaTime);
            drop.color = c;
            if (c.a &lt;= 0f || pos.y &lt; -1400f)
            {
                Destroy(drop.gameObject);
                _goldrain.RemoveAt(i);
                _goldVel.RemoveAt(i);
            }
        }
    }
    // ---- wall face (G07-T007): hold to stack, pour over the top ----
    void BuildWall()
    {
        _wallBuilt = true;
        _siege = new LadderSiege();
        _siege.BeginTeach(); // forced teach drops with the wall (E4 wave2)
        _wallBlock = MakeBlock(_canvas, "WallBlock", TidePlaceholder.Wall(),
            new Vector2(0, -560), 320);
        for (int i = 0; i &lt; _lockLayers.Length; i++)
        {
            // S4 wiring slice a: the lock-arm layers ride the ladder art
            // (art-first) - the human-ladder stack reads as a ladder, the
            // procedural blue block keeps the cadence gates art-agnostic.
            var layer = MakeBlock(_canvas, "LockLayer" + i,
                TidePlaceholder.Face(TidePlaceholder.LadderStem, TidePlaceholder.CrowdBlue, 40),
                new Vector2(0, -760 + i * 52), 40);
            layer.gameObject.SetActive(false); // layers light up as the hold grows
            _lockLayers[i] = layer;
        }
        _wallHint = MakeText(_canvas, "WallHint", "HOLD to stack a human ladder",
            new Vector2(0, -320), 40, TidePlaceholder.GateGold);
        _wallResult = MakeText(_canvas, "WallResult", "", new Vector2(0, -120), 44, TidePlaceholder.GateGold);
    }
    void TickWallHold()
    {
        if (Input.GetMouseButton(0)) _holdHeld += Time.deltaTime;
        int layers = LadderSiege.LayersForHold(_holdHeld);
        for (int i = 0; i &lt; _lockLayers.Length; i++)
            if (_lockLayers[i] != null &amp;&amp; _lockLayers[i].gameObject.activeSelf != (i &lt; layers))
```

```
            _lockLayers[i].gameObject.SetActive(i &lt; layers);
    if (_holdHeld &lt; LadderSiege.StackWindow) return;
    _wallResolved = true;
    WaveSpec spec = WaveTable.SpecFor(FirstWallWave);
    var plan = new RoutePlan { laneUnits = new[] { spec.units }, useLadder = true };
    // Resolve through SYS04 (G07-T008): the resolver folds the captured
    // wave into the shared population counter and answers with the
    // gold-rain triple + evt_gate_captured on this beat.
    AssaultOutcome outcome = _resolver.Resolve(spec, plan);
    if (_wallHint != null) _wallHint.gameObject.SetActive(false);
    if (outcome != AssaultOutcome.Captured) return; // the hold is the only way in - unreachable guard
    if (_wallBlock != null) _wallBlock.color = TidePlaceholder.CrowdBlue; // the tide owns the wall
    if (_wallResult != null) _wallResult.text = "OVER THE TOP - the gate falls from behind";
    if (_population != null) _population.text = _stage.Population.DisplayValue();
    FireCaptureActs();
}
// ---- fork face (G07-T006): stroke to split, slow-mo on the peel ----
void BuildFork()
{
    _forkBuilt = true;
    for (int i = 0; i &lt; 3; i++)
    {
        var gate = MakeBlock(_canvas, "ForkGate" + i, TidePlaceholder.Gate(),
            new Vector2((i - 1) * ForkGateX, -560), 120);
        _forkGates[i] = gate;
        var path = MakeBlock(_canvas, "ForkLane" + i, TidePlaceholder.Lane(),
            new Vector2((i - 1) * ForkGateX, -160), 90);
        path.gameObject.SetActive(true);
    }
    _forkHint = MakeText(_canvas, "ForkHint", "Drag across the lanes to split your tide",
        new Vector2(0, -320), 34, TidePlaceholder.WallSteel);
    _forkResult = MakeText(_canvas, "ForkResult", "", new Vector2(0, -120), 44, TidePlaceholder.GateGold);
}
void TickStroke()
{
    if (Input.GetMouseButtonDown(0))
    {
        _stroking = true;
        AddDwell(Input.mousePosition);
    }
    else if (_stroking &amp;amp;&amp;amp; Input.GetMouseButton(0)) AddDwell(Input.mousePosition);
    else if (_stroking) CommitStroke();
}
void AddDwell(Vector3 screenPos)
{
    int lane = Mathf.Clamp((int)(screenPos.x / Screen.width * 3f), 0, 2);
    _dwell[lane] += Time.deltaTime;
}
void CommitStroke()
{
```

```
    _stroking = false;
    _forkResolved = true;
    WaveSpec spec = WaveTable.SpecFor(FirstPincerWave);
    RoutePlan plan = SplitValve.PlanFromStroke(spec.units, _dwell);
    // Resolve through SYS04 (G07-T008): the resolver answers with the
    // capture triple / loss pair and folds the population.
    AssaultOutcome outcome = _resolver.Resolve(spec, plan);
    if (_forkHint != null) _forkHint.gameObject.SetActive(false);
    // the peel reads in slow motion - the recordable share beat
    if (plan.laneUnits.Length &gt;= 2)
    {
        Time.timeScale = (float)SplitValve.SlowMoScale;
        _slowMoLeft = (float)SplitValve.SlowMoDuration;
    }
    SpreadStream(plan);
    ShowForkResult(spec, plan, outcome);
}
void SpreadStream(RoutePlan plan)
{
    int lanes = Mathf.Clamp(plan.laneUnits.Length, 1, 3);
    for (int i = 0; i &lt; _stream.Count; i++)
    {
        var block = _stream[i];
        if (!block.gameObject.activeSelf) continue;
        var pos = block.rectTransform.anchoredPosition;
        pos.x = LaneX(i % lanes, lanes);
        block.rectTransform.anchoredPosition = pos;
    }
}
void ShowForkResult(WaveSpec spec, RoutePlan plan, AssaultOutcome outcome)
{
    if (outcome == AssaultOutcome.Captured)
    {
        for (int i = 0; i &lt; _forkGates.Length; i++)
            if (_forkGates[i] != null &amp;&amp; plan.laneUnits.Length &gt;= 2)
                _forkGates[i].color = TidePlaceholder.CrowdBlue; // the tide owns the fork
        if (_forkResult != null)
            _forkResult.text = plan.laneUnits.Length &gt;= 2 ? "ENCIRCLED - the gate falls"
                                                            : "the gate falls";
        // population already folded by the resolver into the shared counter
        if (_population != null) _population.text = _stage.Population.DisplayValue();
        FireCaptureActs();
    }
    else if (_forkResult != null)
    {
        _forkResult.text = "Guards converged - the wave is wiped out";
        _forkResult.color = TidePlaceholder.WallSteel;
    }
}
static float LaneX(int lane, int lanes)
```

```
        {
            if (lanes &lt;= 1) return 0f;
            if (lanes == 2) return lane == 0 ? -ForkGateX * 0.75f : ForkGateX * 0.75f;
            return (lane - 1) * ForkGateX;
        }
        static Color HexColor(int hex)
            =&gt; new Color(((hex &gt;&gt; 16) &amp; 0xFF) / 255f, ((hex &gt;&gt; 8) &amp; 0xFF) / 255f, (hex &amp; 0xFF
        Text MakeText(Transform parent, string name, string content, Vector2 anchored, int size, Color color)
        {
            var go = new GameObject(name, typeof(Text));
            go.transform.SetParent(parent, false);
            var text = go.GetComponent&lt;Text&gt;();
            text.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            text.fontSize = size;
            text.color = color;
            text.alignment = TextAnchor.MiddleCenter;
            text.horizontalOverflow = HorizontalWrapMode.Wrap;
            text.verticalOverflow = VerticalWrapMode.Overflow;
            var rt = text.rectTransform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 1f);
            rt.anchoredPosition = anchored;
            rt.sizeDelta = new Vector2(1000, size + 20);
            text.text = content;
            return text;
        }
        Image MakeBlock(Transform parent, string name, Sprite sprite, Vector2 anchored, float size)
        {
            var go = new GameObject(name, typeof(Image));
            go.transform.SetParent(parent, false);
            var image = go.GetComponent&lt;Image&gt;();
            image.sprite = sprite;
            image.SetNativeSize();
            var rt = image.rectTransform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 1f);
            rt.anchoredPosition = anchored;
            var scale = size / Mathf.Max(1f, rt.sizeDelta.x);
            rt.localScale = new Vector3(scale, scale, 1f);
            return image;
        }
    }
}
// WaveTable.cs
// WaveTable.cs - embedded level table for MobTide (G07-T004, requirements section 6 schema).
// The whole difficulty spine is one deterministic rule set (config embedded, zero
// download faces, red line 12): every fourth wave rotates Plain -&gt; Wall -&gt; Plain -&gt;
// Pincer so the first minutes teach all three verbs in a fixed order (E4 Stage1
// timeline), the tide size grows on a five-step cosmetic cycle, and walls keep the
// E4 law that a frontal brute push never wins (guard 999 = ladder is the solution).
using System.Text;
namespace Sy.MobTide
```

```
{
    public enum WaveType { Plain, Wall, Pincer }
    /// &lt;summary&gt;One wave of the assault ladder - pure data, no engine types.&lt;/summary&gt;
    public sealed class WaveSpec
    {
        public int waveIndex;      // 0-based wave number
        public WaveType type;      // choreography family
        public int lanes;          // guarded lanes (1 plain/wall, 3 pincer)
        public int units;          // attacker units this wave
        public double guardPower;  // per-lane defender strength
    }
    /// &lt;summary&gt;Frozen section 6 schema: the residue rules ARE the table.&lt;/summary&gt;
    public static class WaveTable
    {
        public const double PlainGuardPower = 10.0;
        public const double WallBruteGuardPower = 999.0; // frontal push never wins a wall
        public const double PincerGuardPower = 8.0;
        public const int FirstWave = 0;
        public const int LastWave = 199;   // 200-wave bot ladder stays in-table
        /// &lt;summary&gt;The wave for any index: type/lanes by w%4, size by w%5
        /// (malformed negatives normalize, never throw).&lt;/summary&gt;
        public static WaveSpec SpecFor(int waveIndex)
        {
            if (waveIndex &lt; FirstWave) waveIndex = FirstWave;
            int m = waveIndex % 4;
            var s = new WaveSpec
            {
                waveIndex = waveIndex,
                units = 30 + (waveIndex % 5) * 2,   // 30..38 cosmetic tide size
            };
            if (m == 0 || m == 2) { s.type = WaveType.Plain; s.lanes = 1; s.guardPower = PlainGuardPower; }
            else if (m == 1) { s.type = WaveType.Wall; s.lanes = 1; s.guardPower = WallBruteGuardPower; }
            else { s.type = WaveType.Pincer; s.lanes = 3; s.guardPower = PincerGuardPower; }
            return s;
        }
        /// &lt;summary&gt;Canonical signature - the dual-derive byte-parity instrument
        /// (two derives of one wave agree here or nowhere; frozen schema face).&lt;/summary&gt;
        public static string Signature(WaveSpec s)
        {
            if (s == null) return "null";
            var sb = new StringBuilder();
            sb.Append('w').Append(s.waveIndex)
              .Append('|').Append(s.type)
              .Append("|lanes:").Append(s.lanes)
              .Append("|units:").Append(s.units)
              .Append("|guard:").Append(s.guardPower.ToString("0.##"));
            return sb.ToString();
        }
    }
}
```